AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ INFORMATYKI, ELEKTRONIKI i TELEKOMUNIKACJI
INSTYTUT ELEKTRONIKI

PRACA DYPLOMOWA INŻYNIERSKA

System rozpoznający kolory z przeznaczeniem dla niewidomych i robotów

A color recognition system for the blind and robots

Autor: *Maciej Skrobisz*
Kierunek studiów: Elektronika i Telekomunikacja
Opiekun pracy: *prof. Dr hab. Inż. Andrzej Kos*

Kraków, 2026

Spis treści

Spis stosowanych symboli i skrótów: 4

Wstęp 5

Cel Pracy 7

1. Opis dostępnych technik i technologii 8

1.1 Moduł do akwizycji danych 8

1.1.1 Metoda punktowa (kolorymetry/spektrometry) 8

1.1.2 Metoda Obszarowa (kamery CCD/CMOS) 8

1.2 Algorytmy interpretacji i klasyfikacji danych pomiarowych 8

1.2.1 Analiza wektora danych (dla metod punktowych/kolorymetrów) 8

1.2.2 Deterministyczne przetwarzania obrazu (dla metod obszarowych) 9

1.2.3 Uczenie maszynowe i sieci neuronowe (dla metod obszarowych) 9

1.3 Jednostka obliczeniowa i sterująca 10

1.3.1 Podstawowe mikrokontrolery (Arduino/AVR) 11

1.3.2 Wysokowydajne układy SoC/IoT (np. ESP32, STM32) 11

1.3.3 Komputery jednopłytkowe – SBC (np. Raspberry Pi) 12

1.4 Zasilanie układu 12

1.4.1 Zewnętrzny magazyn energii (Powerbank USB) 12

1.4.2 Własny układ z przetwornicą (Li-Po + Boost Converter) 12

1.4.3 Dedykowane moduły i nakładki (np. UPS HAT) 13

1.5 Układ do komunikacji dźwiękowej 13

1.5.1 Programowa generacja PWM (Pulse Width Modulation) 13

1.5.2 Interfejs cyfrowy I2S + dedykowany DAC (Digital-to-Analog Converter) 14

1.5.3 Zewnętrzna karta dźwiękowa USB 14

1.6 Komunikacja M2M (maszyna-maszyna) 14

1.6.1 Transmisja szeregowa UART (Universal Asynchronous Receiver-Transmitter) 14

2. Projekt i implementacja systemu 16

2.1 Platforma sprzętowa (Hardware) 17
2.1.1 Jednostka centralna – Raspberry Pi Zero 2 W 17
2.1.2 Moduł akwizycji obrazu – Raspberry Pi Camera Module 3 18
2.1.3 Zasilanie układu – UPS HAT (C), Waveshare 19
2.1.4 Układ do komunikacji dźwiękowej – MAX98357A + głośnik 40mm 19
2.1.5 Komunikacja M2M – UART wirtualny (USB Gadget) 21
2.1.6 Interfejs analogowy – dwa przyciski monostabilne 22
2.2 Oprogramowanie i algorytmy sterujące 22
2.2.1 Środowisko uruchomieniowe i biblioteki 23
2.2.2 Przygotowanie modelu sieci neuronowej 24
2.2.3 Logika sterująca – implementacja skryptu detekcji kolorów na Raspberry Pi 27
2.2.4 Integracja z systemem operacyjnym (Autostart) 30
3. Weryfikacja i badania eksperymentalne 33
3.1 Testy funkcjonalne 33
3.2 Badanie skuteczności sieci neuronowej 34
3.2.1 Analiza przebiegu uczenia 34
3.2.2 Szczegółowa analiza błędów (macierz pomyłek, raport klasyfikacji) 35
3.2.3 Wpływ kwantyzacji i konwersji TFLite na jakość detekcji 38
3.2.4 Wydajność czasowa i zasobowa systemu 39
4. Podsumowanie i wnioski końcowe 42
4.1 Analiza napotkanych problemów i ograniczeń 42
4.1.1 Ograniczenia optyczne i oświetleniowe 42
4.1.2 Zarządzanie zasobami pamięci RAM i CPU 42
4.1.3 Opóźnienia podsystemu audio i ładowanie systemu operacyjnego 43
4.2 Możliwość dalszego rozwoju projektu 43
4.3 Wnioski końcowe 44
Bibliografia: 45

Spis stosowanych symboli i skrótów:

AI – *Artificial Intelligence* – sztuczna inteligencja

ALSA – *Advanced Linux Sound Architecture* – zaawansowana architektura dźwięku w systemie Linux

ARM – *Advanced RISC Machine* – rodzina architektury procesorów

AWB – *Automatic White Balance* – automatyczny balans bieli

BMS – *Battery Management System* – system zarządzania baterią

CNN – *Convolutional Neural Network* – konwolucyjna sieć neuronowa

CPU – *Central Processing Unit* – centralna jednostka obliczeniowa (procesor)

CRI – *Color Rendering Index* – wskaźnik oddawania barw

GPIO – *General Purpose Input/Output* – uniwersalne wejścia/wyjścia cyfrowe

GPU – *Graphics Processing Unit* – jednostka przetwarzania graficznego

GUI – *Graphical User Interface* – graficzny interfejs użytkownika

I2C – *Inter-Integrated Circuit* – magistrala szeregowa I^2C

I2S – *Inter-IC Sound* – standard interfejsu dźwiękowego

INT8 – *8-bit Integer* – 8-bitowa liczba całkowita

ISP – *Image Signal Processor* – procesor sygnałowy obrazu

JSON – *JavaScript Object Notation* – format wymiany danych JSON

LED – *Light Emitting Diode* – dioda elektroluminescencyjna

M2M – *Machine to Machine* – komunikacja maszyna-maszyna

ML – *Machine Learning* – uczenie maszynowe

MLP – *Multilayer Perceptron* – wielowarstwowy perceptron

OS – *Operating System* – system operacyjny

PCB – *Printed Circuit Board* – obwód drukowany

RAM – *Random Access Memory* – pamięć operacyjna

RGB – *Red, Green, Blue* – model przestrzeni barw RGB

RTOS – *Real-Time Operating System* – system operacyjny czasu rzeczywistego

SSH – *Secure Shell* – protokół komunikacyjny sieciowych powłok

TFLite – *TensorFlow Lite* – lekka wersja biblioteki TensorFlow

TinyML – *Tiny Machine Learning* – uczenie maszynowe na systemach wbudowanych

UART – *Universal Asynchronous Receiver-Transmitter* – uniwersalny asynchroniczny nadajnik-odbiornik

UPS – *Uninterruptible Power Supply* – zasilacz awaryjny

Wstęp

Dla osób widzących, postrzeganie kolorów jest znaczącym elementem dnia codziennego. Oprócz przyjemności płynącej z obserwacji przyrody i barwnych krajobrazów, umiejętność rozróżnienia barwy odgrywa również kluczową rolę w codziennych czynnościach. Sprawdzenie świeżości owoców, rozpoznanie niebezpieczeństwa – na przykład czerwonego światła sygnalizatora, czy dobór odzieży, to zadania wymagające identyfikacji koloru danego obiektu. Utrata tej zdolności prowadzi do wielu trudności i ograniczenia samodzielności.

Szacuje się, że w Polsce żyje około 163,5 tysiąca osób z orzeczoną niepełnosprawnością z powodu dysfunkcji narządu wzroku, z czego u 40% z nich wada jest na tyle znaczna, że wymagają dodatkowego wsparcia w codziennym funkcjonowaniu [1]. Na świecie problem ten dotyczy 295 milionów osób, wliczając w to 43 miliony osób całkowicie niewidomych, a liczba ta stale rośnie [2].

Liczebność tej grupy społecznej prowadzi do podejmowania inicjatyw związanych z projektowaniem urządzeń skierowanych dla osób o szczególnych potrzebach. Kluczową cechą takich produktów musi być maksymalnie uproszczony i intuicyjny interfejs umożliwiający bezproblemową obsługę przez użytkowników w każdym wieku. Ekrany dotykowe oraz aplikacje mobilne mogą być dobrym rozwiązaniem dla młodych lub tych, którzy opanowali ich obsługę przed utratą wzroku. Niestety osoby starsze lub niewidome od urodzenia napotykają znaczne bariery w korzystaniu z tego typu rozwiązań. Wynika stąd zapotrzebowanie na urządzenia o maksymalnie uproszczonej obsłudze, wyposażone w interfejs fizyczny, z którego korzystać będzie mogło zarówno dziecko, jak i osoba w podeszłym wieku, nieobeznana z nowoczesnymi technologiami.

Dlaczego rozpoznawanie kolorów jest na tyle istotne, że uzasadnia budowę dedykowanego urządzenia? Osoby z dysfunkcją wzroku często mają wyostrzone pozostałe zmysły – dotyku, węchu i słuchu. Są w stanie częściowo skompensować brak wzroku, przykładowo wykorzystując dźwięki otoczenia do orientacji przestrzennej lub rozpoznając przedmioty dotykiem. Kolor jest jednak informacją, której nie da się pozyskać za pomocą innych zmysłów. Niemożliwe staje się samodzielne zlokalizowanie plamy na ubraniu, ocena dojrzałości owocu czy identyfikacja nominału banknotu (który często kodowany jest także barwą).

Jakie rozwiązania dostępne są obecnie na rynku? Poza wspomnianymi aplikacjami mobilnymi dostępne są nieliczne urządzenia z interfejsem fizycznym, takie jak: Colorino, ColorTest Standard, czy Color Star firmy CareTec [3]. Produkty te są jednak ograniczone technologicznie, a ich największą wadą jest cena, wahająca się od 1275 zł do nawet 3000 zł [4]. Wysokie koszty dostępnych rozwiązań tworzą istotną barierę finansową dla potencjalnych użytkowników.

Wskazuje to na konieczność opracowania tańszego odpowiednika, wykorzystującego nowoczesne technologie i aktualne możliwości techniki mikroprocesorowej. Realizacja tego zadania stanowi główny cel niniejszej pracy.

Cel Pracy

Celem pracy jest zaprojektowanie i wykonanie systemu identyfikującego kolory w czasie rzeczywistym oraz komunikującego je użytkownikowi w formie komunikatów głosowych lub urządzeniom zewnętrznym za pomocą interfejsu cyfrowego.

Kluczowymi cechami opracowywanego urządzenia są:

- Mobilność i zasilanie bateryjne umożliwiające niezależną pracę w terenie.
- Prosty, fizyczny interfejs niewymagający kontroli wzrokowej, dostosowany do potrzeb osób z dysfunkcją wzroku.
- Skuteczność działania w zmiennych warunkach oświetleniowych.
- Wydajne przetwarzanie obrazu w trybie ciągłym, pozwalające na płynną analizę otoczenia.

1. Opis dostępnych technik i technologii

W tym rozdziale przedstawię główne elementy i techniki wykorzystywane w projekcie. Opiszę tok działań i analiz prowadzących do końcowej implementacji systemu oraz nakreślę funkcjonalność poszczególnych jego modułów.

1.1 Moduł do akwizycji danych

Istnieje wiele technik umożliwiających pobranie z otoczenia danych potrzebnych do skutecznej identyfikacji koloru, a także jego nasycenia i jasności. Można podzielić je na dwie kategorie:

1.1.1 Metoda punktowa (kolorymetry/spektrometry)

Precyzyjna, ale wymagająca ściśle kontrolowanych warunków pomiarowych lub bezpośredniego styku z badaną powierzchnią do skutecznej analizy. Pomiar na mikroskopijnym obszarze może dać błędny wynik w przypadku napotkania lokalnego przebarwienia, zanieczyszczenia, czy drobnej niejednorodności faktury obiektu.

1.1.2 Metoda Obszarowa (kamery CCD/CMOS)

Lepiej naśladuje działanie ludzkiego oka. Pozwala na kontekstową analizę danych, niewymagającą szczególnie przygotowanego środowiska pomiaru. Dopuszcza mniej precyzyjne stosowanie urządzenia – zamiast przykładać czujnik idealnie do powierzchni, wystarczy jedynie skierować kamerę w stronę badanego obiektu z niewielkiej, nieprecyzyjnie określonej odległości. Metoda ta charakteryzuje się większą tolerancją na błędy pozycjonowania oraz lepiej generalizuje np. nierówne tekstury przedmiotu.

1.2 Algorytmy interpretacji i klasyfikacji danych pomiarowych

Wybór metody pozyskania danych determinuje sposób ich późniejszego przetwarzania. System może otrzymywać proste wartości liczbowe (np. z czujników punktowych) lub złożone macierze pikseli (z kamery). Konieczne jest zastosowanie odmiennych algorytmów klasyfikacyjnych w przytoczonych przypadkach. W procesie projektowania układu rozważono trzy główne podejścia do interpretacji danych:

1.2.1 Analiza wektora danych (dla metod punktowych/kolorymetrów)

Jest to metoda dla czujników podających wynik w postaci liczbowej. Klasyfikacja w takim przypadku polega na porównaniu matematycznym, najczęściej poprzez obliczenie odległości euklidesowej w przestrzeni trójwymiarowej pomiędzy wektorem zmierzonym, a wektorami

wzorcowymi zapisanymi w bazie danych. Prostota i znikome wymagania pamięciowe tego rozwiązania są niestety niwelowane przez wymóg idealnych warunków pomiarowych. Jakiekolwiek czynniki zewnętrzne nieprzewidziane w procesie tworzenia bazy danych, powodują znaczne błędy pomiarowe, a w wyniku nieskuteczną klasyfikację koloru.

1.2.2 Deterministyczne przetwarzania obrazu (dla metod obszarowych)

W przypadku wykorzystania kamer, dane wejściowe mają postać dwuwymiarowej macierzy pikseli. Klasycznym podejściem jest zastosowanie algorytmów filtrujących np. konwersji do przestrzeni barw HSV (Hue, Saturation, Value) i wykorzystanie progowania (thresholding). Wycinane są z obrazu fragmenty mieszczące się w zadanym zakresie składającym się z:

- Odcienia na kole barw (H): 0-360°,
- Nasycenia – intensywności koloru (S): 0-100%,
- Wartości – intensywności światła (V): 0-100%.

Podobnie do poprzedniej metody, zaletą jest szybkość działania, lecz niska odporność na zakłócenia wizualne (cienie, odblaski), pomimo zastosowania filtrów i przetwarzania, nie gwarantuje stabilnych i powtarzalnych rezultatów.

1.2.3 Uczenie maszynowe i sieci neuronowe (dla metod obszarowych)

W przeciwieństwie do metod deterministycznych, rozwiązania te nie polegają na sztywnym programowaniu reguł, lecz na tworzeniu modelu matematycznego na podstawie danych treningowych. W kontekście projektu i klasyfikacji kolorów wyróżnić można trzy typy rozwiązań:

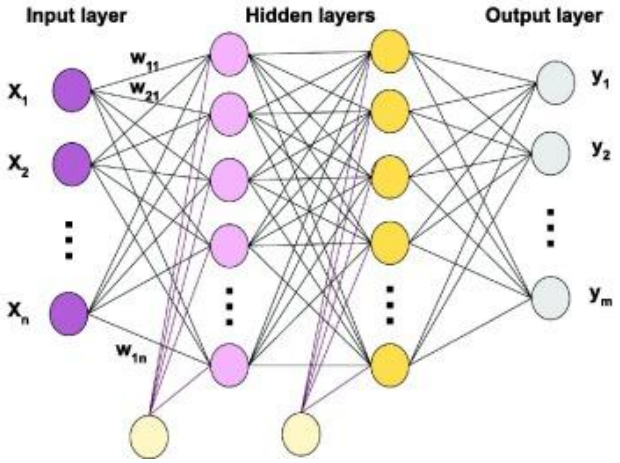
- **Klasyczne algorytmy klasyfikacyjne (`k-NN`, `SVM`)**

  Algorytm k-najbliższych sąsiadów (`k-NN`) zapamiętuje próbki treningowe w wielowymiarowej przestrzeni cech, a testowana próbka danych klasyfikowana jest po znalezieniu k najbliższych punktów wzorcowych. Maszyna Wektorów Nośnych (`SVM`) wyznacza optymalną hiperpłaszczyznę oddzielającą różne klasy kolorów z jak największym marginesem separacji. Metody te nie pracują zazwyczaj bezpośrednio na obrazie, lecz wymagają wcześniejszej ekstrakcji cech, czyli obliczenia parametrów opisujących dany obraz, np. histogramu barw.

- **Standardowe sieci neuronowe (`ANN` / `MLP`)**

  Sieci `MLP` (Multi-Layer Perceptron) zbudowane są z warstw neuronów połączonych w strukturze „każdy z każdym”. Niestety do analizy obrazu tą metodą musi zostać on „spłaszczony” (flattening) do jednego długiego wektora pikseli. Powoduje to niską

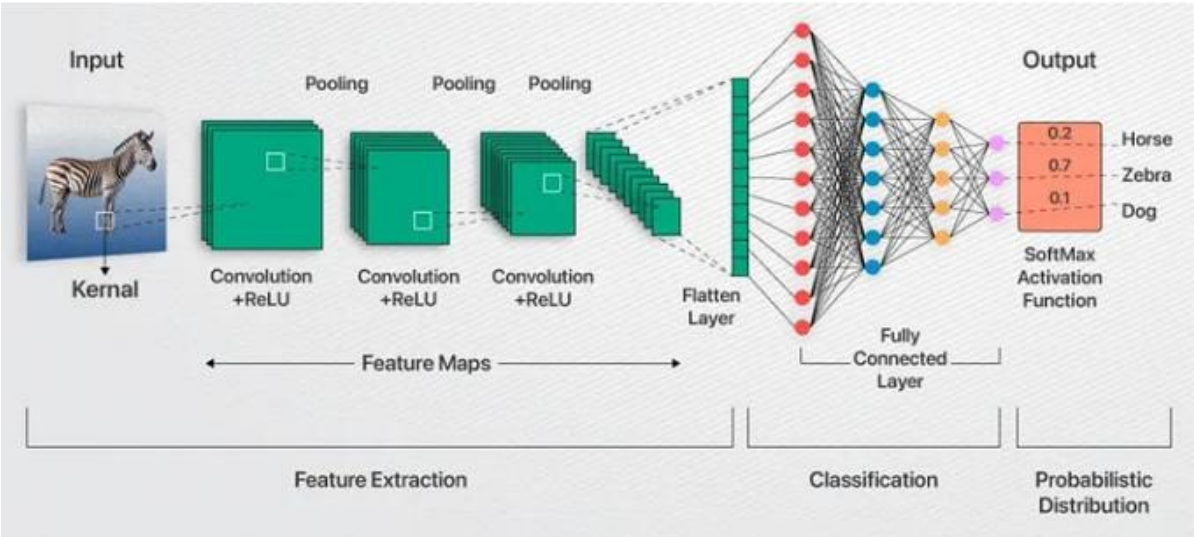
odporność na przesuwanie obiektu w kadrze, co jest uciążliwe w systemie czasu rzeczywistego (Real-Time).



Rys. 1. Topologia sieci MLP [5]

- **Konwolucyjne sieci neuronowe (CNN)**

  Jest to wyspecjalizowana architektura sieci neuronowych do przetwarzania danych o strukturze siatki, np. obrazów. CNN automatycznie uczy się, które cechy obrazu są ważne dla rozpoznania – w przypadku tego projektu: koloru. W przeciwieństwie do algorytmów klasycznych (k-NN, SVM), zamiast polegać na ściśle zaprogramowanych przez użytkownika cechach, sieć sama analizuje wpływ m.in. odblasków i cieni na skuteczność klasyfikacji [6].



Rys. 2. Schemat Konwolucyjnej Sieci Neuronowej [7]

1.3 Jednostka obliczeniowa i sterująca

Najistotniejszym elementem do wyboru jest jednostka centralna odpowiadająca za sterowanie pozostałymi modułami systemu oraz dostarczenie wystarczającej mocy obliczeniowej

dla przeprowadzanych operacji. Musi ona zapewniać odpowiednie interfejsy (np. CSI, UART) gwarantujące kompatybilność z pozostałymi blokami funkcjonalnymi. Kluczowa jest także dostępność zaawansowanych bibliotek wspomagających wykonywane przez układ operacje przetwarzania danych. Do dostępnych na rynku platform sterujących należą:

1.3.1 Podstawowe mikrokontrolery (Arduino/AVR)

Reprezentują grupę układów o najprostszej architekturze, dedykowanych do nieskomplikowanych zadań sterowania.

- **Charakterystyka:** Układy o niewielkim poborze prądu i małej mocy obliczeniowej (częstotliwości sięgające kilkudziesięciu MHz). Nie posiadają systemu operacyjnego, wykonują jeden program w pętli (Bare Metal).
- **Zalety:** Niski koszt, natychmiastowe działanie, prosta obsługa nieskomplikowanych czujników i sygnałów (np. przycisków).
- **Wady:** Znikoma pojemność pamięci operacyjnej (zwykle rzędu kilku kB), uniemożliwiająca wczytywanie zdjęć do pamięci i operacje na nich. Brak natywnej obsługi kamer oraz plików audio wysokiej jakości.

1.3.2 Wysokowydajne układy SoC/IoT (np. ESP32, STM32)

Stanowią one kompromis pomiędzy energooszczędnością mikrokontrolerów a wydajnością pełnych systemów komputerowych.

- **Charakterystyka:** Posiadają 32-bitowe procesory o taktowaniach sięgających 240 MHz. Często wyposażone są we wbudowane moduły Wi-Fi/Bluetooth oraz interfejsy obsługujące małe kamery (np. OV2640).
- **Zalety:** Zdolność do przetwarzania obrazu, niska cena, energooszczędność. Istnienie wersji bibliotek AI oraz ML (np. TensorFlow Lite Micro) przeznaczonych na te układy. Dostępne w najnowszych modelach (np. ESP32-S3) instrukcje wektorowe (SIMD), zapewniające sprzętowe wsparcie dla obliczeń macierzowych wykorzystywanych w sieciach neuronowych [8].
- **Wady:** Wymóg jednoczesnego odtwarzania dźwięku i analizy obrazu może graniczyć z możliwościami obliczeniowymi układów lub wprowadzać znaczne opóźnienia. Niepełne biblioteki oraz niższe możliwości związane z AI/ML.

1.3.3 Komputery jednopłytkowe – SBC (np. Raspberry Pi)

Są to kompletne jednostki obliczeniowe, oferujące pełną funkcjonalność komputera osobistego w zminiaturyzowanej formie.

- **Charakterystyka:** Pełnoprawne komputery z systemami operacyjnymi (najczęściej Linux), procesorem wielordzeniowym i dużą pojemnością pamięci RAM (512 MB i więcej).
- **Zalety:** Obsługa języków wysokiego poziomu (Python) i pełnych bibliotek ML (OpenCV, TensorFlow). Wielowątkowość umożliwia płynne działanie kamery, przetwarzanie obrazu i dźwięku jednocześnie.
- **Wady:** Większy pobór prądu, dłuższy czas potrzebny na uruchomienie systemu.

1.4 Zasilanie układu

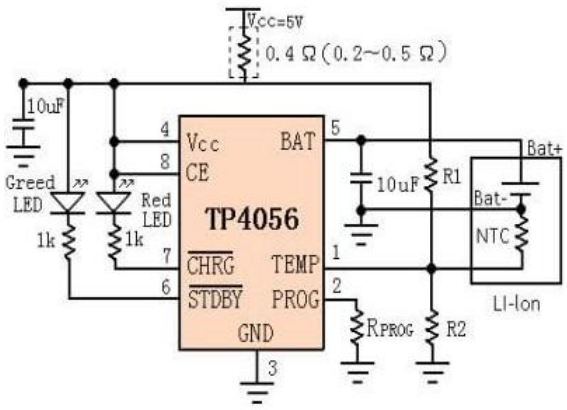
Zapewnienie mobilności projektowanego urządzenia wymaga wykorzystania niezależnego źródła energii. Priorytetem jest kompaktowość systemu oraz prostota użytkowania, zatem do rozważanych opcji należą:

1.4.1 Zewnętrzny magazyn energii (Powerbank USB)

Pozwoli on na zminimalizowanie rozmiaru samego urządzenia oraz wykorzystanie dużych pojemności baterii, rzędu 10 000 mAh, które oferowane są przez miniaturowe modele dostępne na rynku. Największą wadą takiego rozwiązania jest potrzeba stałego podłączenia powerbanku, a obecność przewodu znacznie utrudnia swobodne manewrowanie urządzeniem.

1.4.2 Własny układ z przetwornicą (Li-Po + Boost Converter)

Rozwiązanie typu DIY, wykorzystujące ogniwo Li-Ion/Li-Po, osobny moduł ładowania (np. TP4056) oraz przetwornicę Step-up zapewniającą napięcie 5 V. Mimo niskiej ceny, układ taki wraz z okablowaniem zajmuje dużo miejsca i często generuje zakłócenia elektromagnetyczne, mogące negatywnie wpływać na pracę pozostałych elementów. Trudności w zintegrowaniu zabezpieczeń przed głębokim rozładowaniem oraz często występujące elementy niskiej jakości mogą stanowić zagrożenie pożarowe, co wymaga szczegółowej analizy i uwzględnienia w procesie projektowania.



Rys. 3. Schemat układu z modułem ładowania TP4056 [9]

1.4.3 Dedykowane moduły i nakładki (np. UPS HAT)

Zintegrowane moduły korzystające bezpośrednio z łączy dostępnych na płytce mikrokontrolera, zaprojektowane z myślą o konkretnym jego modelu. Ich budowa często jest dopasowana mechanicznie do układu docelowego, dzięki czemu stanowi bardzo kompaktowe rozwiązanie. Gwarancja kompatybilności oraz opracowane procedury komunikacji i zabezpieczenia są kluczowe dla projektanta systemu. Największym mankamentem tego wariantu jest cena, wyższa od pozostałych rozwiązań.

1.5 Układ do komunikacji dźwiękowej

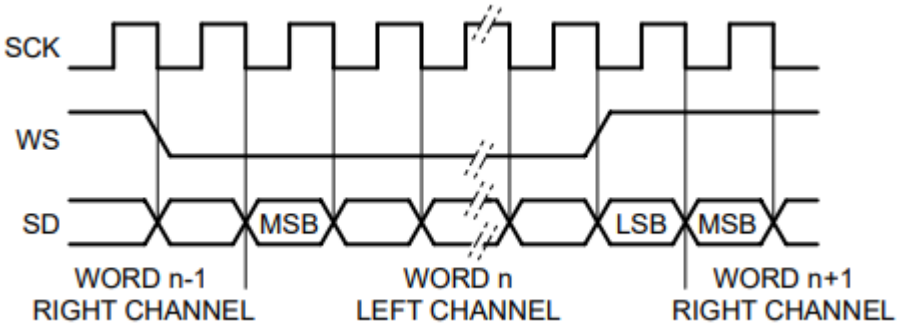
Kluczowym elementem interfejsu w urządzeniu przeznaczonym dla osób z dysfunkcją wzroku jest układ generujący komunikaty głosowe. Muszą one być zrozumiałe nawet w bardzo głośnym środowisku, zatem kluczowa w doborze rozwiązania jest czystość i głośność wytwarzanego dźwięku. W kolejnych podrozdziałach opisano możliwe do wykorzystania w projekcie metody.

1.5.1 Programowa generacja PWM (Pulse Width Modulation)

Niemalże wszystkie najnowsze mikrokontrolery i minikomputery posiadają wbudowaną możliwość korzystania z generowania sygnału (np. audio) poprzez sterowanie szerokością impulsów oraz wygładzanie ich filtrami – PWM. Metoda ta nie wymusza prawie żadnego wkładu pieniężnego – zwykle wystarczy zakup małego głośnika. Niestety jakość dźwięku oferowanego przez to rozwiązanie jest niska, często na tyle, że trudne jest zrozumienie komunikatu imitującego głos człowieka.

1.5.2 Interfejs cyfrowy I2S + dedykowany DAC (Digital-to-Analog Converter)

Interfejs I2S (Inter-IC Sound) służy do przesyłania dźwięku w postaci cyfrowej bezpośrednio z procesora/mikrokontrolera do zewnętrznego przetwornika cyfrowo-analogowego (np. MAX98357A, PCM5102). Dekodowanie zachodzi poza procesorem, co znacząco redukuje szumy zasilania. Koszt małego głośnika i układu DAC jest bardzo niski. Wadami takiego rozwiązania jest zajmowanie pinów GPIO oraz konieczność ręcznego połączenia elementów. Obciążenie CPU jest pomijalne dzięki wykorzystaniu mechanizmu DMA (Direct Memory Access).



Rys. 4. Diagram czasowy dla linii I2S [10]

1.5.3 Zewnętrzna karta dźwiękowa USB

Połączenie standardowej karty dźwiękowej, często przez adapter microUSB-OTG. Jest to najłatwiejsze do implementacji rozwiązanie i oferuje dobrą jakość dźwięku. Niestety gabaryty układu karty z głośnikiem są ogromne jak na wykorzystanie w projekcie mobilnym. Wykorzystywany przez to rozwiązanie port USB znacznie ogranicza rozbudowę systemu ze względu na małą dostępność interfejsów USB Host w większości prostych układów mikrokontrolerowych.

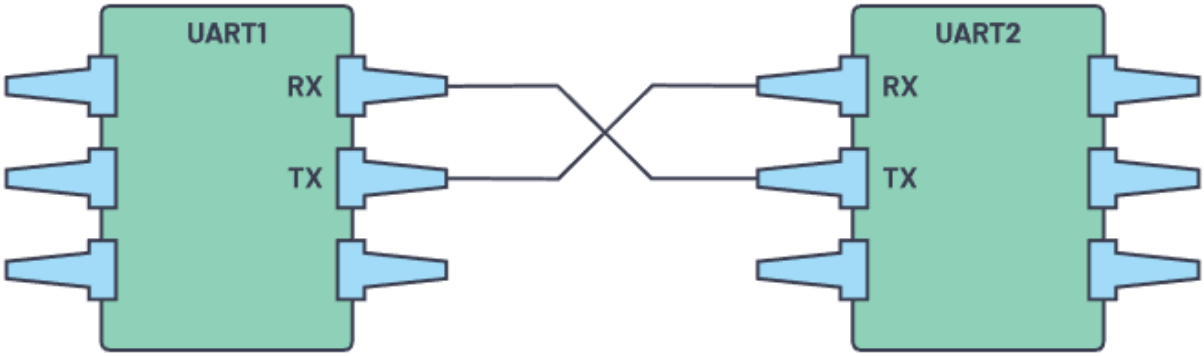
1.6 Komunikacja M2M (maszyna-maszyna)

Oprócz funkcji asystującej osobom z dysfunkcją wzroku, projektowany system ma mieć możliwość komunikacji z maszyną (np. robotem). W takim scenariuszu kluczowa jest nie forma akustyczna, a cyfrowa transmisja wyników działania w formacie zrozumiałym dla sterowników logicznych. Możliwe do użycia kanały komunikacji opisano w kolejnych podrozdziałach.

1.6.1 Transmisja szeregowa UART (Universal Asynchronous Receiver-Transmitter)

To standard w świecie robotyki, obsługiwany przez każdy mikrokontroler i SBC. Polega na asynchronicznej wymianie danych w pełnym dupleksie (Full Duplex – jednocześnie w obu

kierunkach) za pomocą linii sygnałowych TX (nadawanie) oraz RX (odbiór). Nie wymaga on skomplikowanego stosu protokołów ani połączenia z siecią. Jest to korzystne rozwiązanie ze względu na brak narzutu protokołów sieciowych (TCP/IP), bezpośrednie połączenie systemu z układem zewnętrznym oraz determinizm na poziomie warstwy sprzętowej transmisji – dokładnie określony czas przesłania bajtu.



Rys. 5. Schemat komunikacji dwóch interfejsów UART [11]

1.6.2 Komunikacja bezprzewodowa (Wi-Fi / Bluetooth)

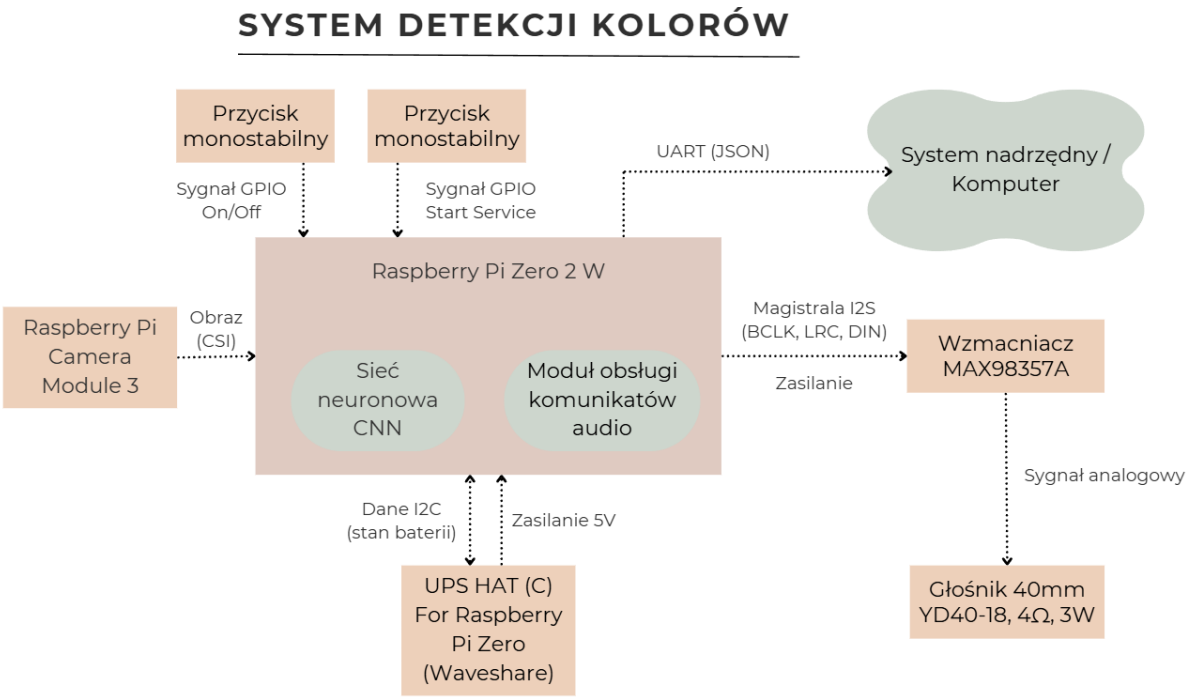
Alternatywą dla połączenia kablowego jest wykorzystanie modułów Bluetooth/Wi-Fi, często wbudowanych w układy mikrokontrolerów i SBC.

- **Wi-Fi (WLAN):** Umożliwia przesyłanie danych w sieci lokalnej z wykorzystaniem protokołów IoT, takich jak MQTT (Message Queuing Telemetry Transport) lub WebSocket. Pozwalają one na dwukierunkową komunikację pomiędzy urządzeniem a aplikacją użytkownika.
- **Bluetooth:** Polega na bezpośrednim parowaniu z robotem lub telefonem. Wykorzystanie profilu SPP (Serial Port Profile) pozwala na emulację połączenia szeregowego (UART) drogą radiową.

Zaletami łączności bezprzewodowej są całkowita mobilność i izolacja galwaniczna układów. Największym problemem tego rozwiązania jest wyższe opóźnienie, które w pracy RT (Real-Time) jest niebagatelne, oraz podatność na zakłócenia ze środowiska.

2. Projekt i implementacja systemu

Architekturę zaprojektowanego systemu przedstawiono na poniższym schemacie blokowym (**Rys. 6**). Prezentuje on użyte elementy, a także kierunek przepływu danych między poszczególnymi modułami.



Rys. 6. Schemat blokowy systemu detekcji kolorów dla osób niewidomych i robotów

Centralnym elementem układu jest minikomputer **Raspberry Pi Zero 2 W**. Odpowiada on za sterowanie wszystkimi wykonywanymi operacjami. System pobiera dane wejściowe z trzech źródeł:

- **Obraz** z kamery Raspberry Pi Camera Module 3 (interfejs CSI).
- **Sygnały sterujące** z przycisków monostabilnych (GPIO).
- **Zasilanie 5 V** oraz parametry telemetryczne z modułu UPS HAT (C) poprzez magistralę I2C.

Następnie informacje przetwarzane są za pomocą sieci neuronowej CNN (klasyfikacja koloru) oraz modułu obsługi audio. Wyniki działania kierowane są do odpowiednich układów wyjściowych:

- **Komunikaty dźwiękowe** – magistralą I2S do wzmacniacza MAX98357A i głośnika 40 mm.
- **Dane cyfrowe** – przesyłane przewodowo w formacie JSON do zewnętrznego komputera z wykorzystaniem protokołu UART.

Szczegółowy opis poszczególnych bloków funkcjonalnych przedstawiono w dalszej części rozdziału.

2.1 Platforma sprzętowa (Hardware)

2.1.1 Jednostka centralna – Raspberry Pi Zero 2 W

Jako fizyczną bazę urządzenia wybrano minikomputer SBC **Raspberry Pi Zero 2** w [12]. Moc obliczeniowa tego modelu znacząco przewyższa możliwości mikrokontrolerów, takich jak ESP32-S3, oferując czterordzeniowy procesor 64-bit Arm Cortex-A53 o taktowaniu 1 GHz w porównaniu do układu dwurdzeniowego (240 MHz). W połączeniu z kompaktowym rozmiarem i niską ceną sprawia to, że moduł ten najlepiej realizuje założenia projektowe. Kluczowymi atutami przesądzającymi o tym wyborze były także:

- Obecność sprzętowego procesora obrazu (ISP) na złączu CSI.
- Kompatybilność z modułami kamer wysokiej jakości.
- Możliwość wykorzystania języka Python oraz zaawansowanych bibliotek (NumPy, TensorFlow, OpenCV).
- Duża liczba wyprowadzeń GPIO umożliwiająca rozbudowę projektu.

Istotnym ograniczeniem zastosowania minikomputera jest wyższy pobór mocy w porównaniu do rozwiązań opartych na mikrokontrolerach.

Jako nośnik pamięci masowej wykorzystano kartę pamięci microSD o pojemności 32 GB (wersja certyfikowana dla Raspberry Pi). Zapewnia to wystarczającą przestrzeń na pliki systemowe, biblioteki programistyczne oraz zbiory danych.

Przygotowanie minikomputera zaczęto od instalacji systemu operacyjnego 64-bit Raspberry Pi OS Lite - dedykowanej dla tej platformy dystrybucji systemu Linux. Wersja ta nie zawiera GUI składającego się ze środowiska graficznego i menedżera okien, co znacznie obniża spoczynkowe użycie pamięci RAM i CPU. Do przygotowania nośnika z systemem z poziomu komputera PC posłużył program Raspberry Pi Imager. Pozwolił on na wstępną konfigurację parametrów pracy w trybie *headless* (bez podłączonego bezpośrednio monitora i klawiatury), w tym zdefiniowanie nazwy hosta, danych logowania oraz konfigurację połączenia z siecią Wi-Fi. Po zakończeniu instalacji i pierwszym uruchomieniu, dalsza konfiguracja urządzenia odbywała się zdalnie poprzez protokół SSH [13].

Za pomocą narzędzia `raspi-config` oraz edycji pliku tekstowego `/boot/firmware/config.txt` aktywowano niezbędne interfejsy sprzętowe wykorzystywane w projekcie:

- **Interfejs kamery** (automatyczna detekcja modułu Camera Module 3 na złączu CSI).
- **Magistralę I2C** (do odczytu parametrów telemetrycznych z nakładki zasilającej UPS HAT).

- **Interfejs audio I2S** (uruchomienie nakładki `hifiberry-dac` oraz `i2s-mmap` do obsługi wzmacniacza MAX98357A).
- **Port szeregowy UART** (dezaktywacja konsoli systemowej na porcie szeregowym, aby umożliwić czystą transmisję danych JSON).

2.1.2 Moduł akwizycji obrazu – Raspberry Pi Camera Module 3

Jako sensor wizyjny wybrano **Raspberry Pi Camera Module 3**, wyposażony w sensor Sony IMX708 o rozdzielczości 12 Mpx [14]. Kamera zapewnia intuicyjną i prostą obsługę systemu przez osoby niewidome oraz pozwala na większą tolerancję błędów w kadrowaniu (mniej precyzyjne nakierowanie urządzenia). W interpretacji koloru kluczowy jest wysokiej jakości obraz wejściowy, a gwarantują go następujące funkcjonalności:

- **Obsługa HDR (High Dynamic Range)** – szeroki zakres dynamiki jest niezbędny do rejestracji kolorów o wysokiej wierności w trudnych warunkach oświetleniowych.
- **Auto-Focus (AF)** – mechanizm automatycznego ustawiania ostrości, krytyczny w przypadku użytkowania kamery przez osobę niewidomą, niebędącą w stanie precyzyjnie kontrolować odległości od przedmiotu do obiektywu. Jest to również istotne w kontekście wykorzystania sieci CNN, która korzysta także z informacji o strukturze obrazu.

Ponieważ kamera jest głównym czynnikiem limitującym skuteczność działania w projektowanym układzie, uzasadnione jest wykorzystanie urządzenia wyższej klasy. Tańsze kamery nieposiadające AF lub HDR (np. OV5640, IMX219) znacznie pogarszają końcową jakość wyników klasyfikacji.

Połączenie z jednostką centralną zrealizowano za pomocą interfejsu CSI-2 (Camera Serial Interface), który cechuje się znacznie mniejszym opóźnieniem i obciążeniem procesora w porównaniu do kamer USB, dzięki bezpośredniemu wykorzystaniu wbudowanego w procesor układu ISP. Do komunikacji użyto dedykowanej taśmy FFC o zwężonym rastrze pinów (kompatybilnej ze złączem Mini CSI). Moduł osadzony został trwale w obudowie urządzenia.

Integracja modułu kamery z systemem operacyjnym nie wymagała instalacji zewnętrznych sterowników producenta. Obsługa sensora realizowana jest przez mechanizm Device Tree Overlay. W pliku konfiguracyjnym `/boot/firmware/config.txt` ustawiono parametr `camera_auto_detect=1`. Jest on odpowiedzialny za automatyczne wykrycie kamery i załadowanie odpowiednich modułów jądra podczas uruchamiania systemu (boot).

Ze względu na współdzielenie pamięci w układzie SoC, zarezerwowano 128 MB pamięci operacyjnej na potrzeby buforowania strumienia wideo. Wykorzystano do tego wpis

`dtoverlay=vc4-kms-v3d,cma-128`, zmieniający ustawienia alokatora pamięci CMA (Contiguous Memory Allocator).

2.1.3 Zasilanie układu – UPS HAT (C), Waveshare

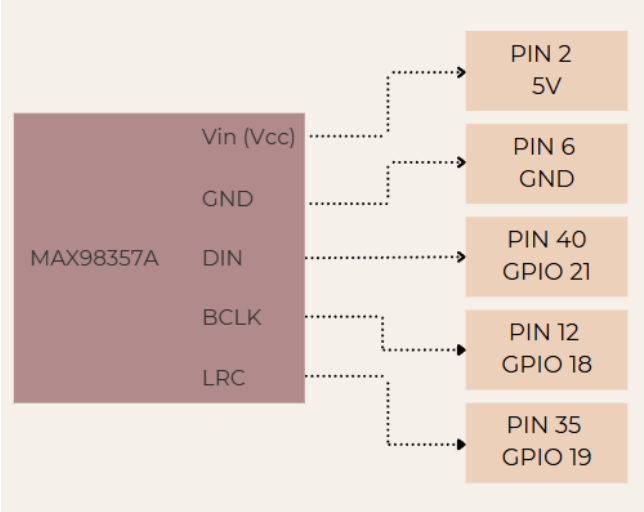
Do zasilania układu zastosowano nakładkę **UPS HAT (C)** producenta Waveshare [15]. Jest to moduł wysoce kompatybilny z Raspberry Pi Zero 2 w – zajmuje minimalną ilość miejsca, zapewnia stabilne napięcie 5 V oraz możliwość komunikacji telemetrycznej przez magistralę I2C. Posiada on również wbudowany kontroler ładowania ETA6003, układ zabezpieczający S8261 oraz inne systemy gwarantujące niezawodne działanie. W projekcie użyto akumulatora litowo-polimerowego (oznaczenie 803040) o napięciu 3,7 V i pojemności 1000 mAh. Przetwornica zintegrowana na płytce UPS HAT podnosi napięcie z 3,7 V na 5 V ze sprawnością ok. 90%.

2.1.4 Układ do komunikacji dźwiękowej – MAX98357A + głośnik 40mm

Jako układ audio wykorzystano wzmacniacz klasy D – MAX98357A, zapewniający wysoką sprawność energetyczną (ok. 90-92%) oraz wystarczająco dobrą jakość do generowania komunikatów głosowych [16]. Do odtwarzania dźwięku służy głośnik YD40-18 z membraną plastikową na resorze gumowym o parametrach:

- Średnica: 40 mm
- Moc: 3 W
- Impedancja: 4 Ω

Elementy te tworzą tani tor audio o parametrach wystarczających do pracy w zróżnicowanych środowiskach akustycznych. Montaż układu wykonano ręcznie za pomocą stacji lutowniczej. Sposób podłączenia do jednostki centralnej przedstawiono na schemacie (**Rys. 7**).



Rys. 7. Schemat blokowy połączeń wzmacniacza MAX98357A z Raspberry Pi Zero 2 W

Opis połączeń:

- Vin (Vcc) – Zasilanie 5 V pobierane z pinu 2.
- GND – Masa układu podłączona do pinu 6 (Ground).
- DIN – Podłączony do GPIO 21 (PCM DOUT). Linia danych szeregowych, którą przesyłane są próbki dźwięku z Raspberry Pi do wzmacniacza.
- BCLK – Podłączony do GPIO 18 (PCM CLK). Sygnał zegara bitowego, który wyznacza moment odczytu każdego pojedynczego bitu danych przesyłanych linią DIN.
- LRC – Podłączony do GPIO 19 (PCM FS). Sygnał Word Select, który informuje wzmacniacz, czy przesyłane dane należą do lewego, czy prawego kanału.

Aby system operacyjny poprawnie rozpoznał zewnętrzny DAC MAX98357A, aktywowano nakładkę sprzętową (Device Tree Overlay) w pliku `/boot/firmware/config.txt` za pomocą wpisu: `dtoverlay=hifiberry-dac`. Konfiguracja ta przestawia piny GPIO w tryb dedykowany dla interfejsu I2S.

Użyty wzmacniacz nie posiada sprzętowej kontroli wzmocnienia (potencjometru), zatem do kontroli głośności dźwięku stworzono wirtualny kontroler w warstwie programowej ALSA (Advanced Linux Sound Architecture). W pierwszej kolejności do pliku konfiguracyjnego config.txt dopisano: `dtoverlay=i2s-mmap`. Zapewnia to kompatybilność z bibliotekami odtwarzania dźwięku wykorzystującymi mapowanie pamięci. Następnie utworzono plik konfiguracyjny `/etc/asound.conf`, którego treść zdefiniowała wtyczkę softvol, przechwytującą strumień audio przed wysłaniem go do fizycznej karty dźwiękowej (`card 1`). Konfigurację przedstawiono na **Rys. 8**.

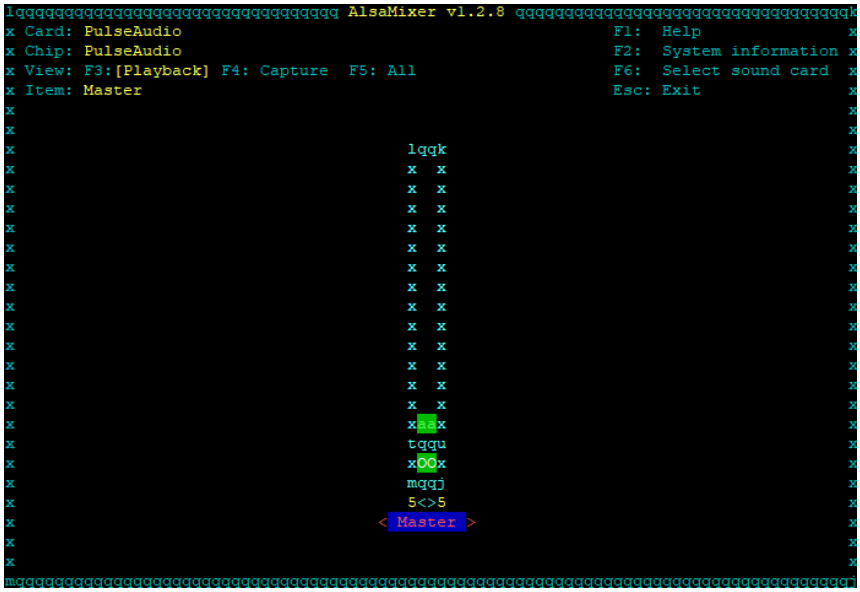
```
pcm.!default {
    type plug
    slave.pcm "softvol"
}

pcm.softvol {
    type softvol
    slave {
        pcm "plughw:1"
    }
    control {
        name "Master"
        card 1
    }
}

ctl.!default {
    type hw
    card 1
```

Rys. 8. Zawartość pliku konfiguracyjnego /etc/asound.conf

Powyższa konfiguracja tworzy nowy element sterujący o nazwie `Master`, dzięki czemu możliwe stało się użycie narzędzia `alsamixer` do płynnej regulacji poziomu wyjściowego.



Rys. 9. Wirtualny suwak do kontroli poziomu sygnału elementu `Master`

Aby zapisać ustawiony poziom głośności i przywracać go automatycznie przy każdym starcie systemu, użyto polecenia: `sudo alsactl store`.

2.1.5 Komunikacja M2M – UART wirtualny (USB Gadget)

W celu uproszczenia integracji z systemem nadrzędnym (robotem lub komputerem PC) i minimalizacji okablowania, zrezygnowano z klasycznego interfejsu UART na pinach GPIO, a zastosowano wirtualny port szeregowy realizowany przez złącze micro-USB (USB Gadget Mode) [17]. Rozwiązanie to pozwala na jednoczesne zasilanie układu oraz dwukierunkową transmisję danych za pomocą jednego przewodu.

W celu aktywacji tej funkcjonalności przełączono kontroler USB minikomputera w tryb urządzenia peryferyjnego. Dokonano tego w pliku konfiguracyjnym `/boot/firmware/config.txt`, dodając na końcu wpis: `dtoverlay=dwc2,dr_mode=peripheral`.

Domyślnie Raspberry Pi używa sterownika USB zoptymalizowanego pod działanie jako host, a wykorzystany dwc2 (DesignWare Controller 2) pozwala urządzeniu działać również jako *Gadget* obsługujący pełną specyfikację OTG (On-The-Go). Następnie, w pliku parametrów startowych jądra `/boot/firmware/cmdline.txt`, od razu po słowie `rootwait` dopisano: `Modules-load=dwc2,g_serial`. Moduł `g_serial` odpowiada za emulację łącza szeregowego (CDC ACM), dzięki czemu urządzenie po podłączeniu do portu USB hosta wykrywane jest jako wirtualny port COM (w systemie Linux: `/dev/ttyACM0`). Aby zapobiec konfliktowi,

w którym system operacyjny przechwytywałby komunikaty przeznaczone dla aplikacji sterującej, wyłączono usługę systemową `serial-getty` dla utworzonego portu wirtualnego. Wykonano to poleceniami:

```
sudo systemctl stop serial-getty@ttyGS0.service
sudo systemctl disable serial-getty@ttyGS0.service
```

Dzięki temu kanał `/dev/ttyGS0` jest całkowicie wolny od procesów systemowych i przeznaczony wyłącznie na użytek aplikacji detekcji kolorów.

2.1.6 Interfejs analogowy – dwa przyciski monostabilne

Do manualnej obsługi systemu wykorzystywane są dwa przyciski monostabilne. Podłączone zostały zgodnie z poniższą tabelą **(Tabela 1)**:

Tabela 1. Konfiguracja podłączenia przycisków sterujących

Komponent	**Pin RPi (Sygnał)**	**Pin RPi (Masa/GND)**	**Funkcja**
Przycisk 1	**Pin 5** (GPIO 3)	**Pin 9** (GND)	Włączanie / Wyłączanie (Power)
Przycisk 2	**Pin 11** (GPIO 17)	**Pin 14** (GND)	Detekcja kolorów (Aktywacja skryptu)

Pierwszy z nich odpowiada za wyłączanie i włączanie minikomputera, a drugi za obsługę skryptu Python, który realizuje pełną funkcjonalność urządzenia – obsługę kamery, przetwarzanie danych i odczyt wyników lub przesłanie ich za pomocą UART. Obsługę „Przycisku 1" uruchomiono, dodając na koniec pliku `/boot/firmware/config.txt` wpis: `Dtoverlay=gpio-shutdown,gpio_pin=3`. Dzięki tej konfiguracji zwarcie pinu 5 do masy powoduje bezpieczne zamknięcie systemu lub wybudzenie urządzenia ze stanu uśpienia. Drugi przycisk obsługiwany jest w pełni programowo, z wykorzystaniem biblioteki RPi.GPIO oraz przerwań systemowych.

2.2 Oprogramowanie i algorytmy sterujące

Warstwa programowa stanowi element spajający wszystkie omówione wcześniej moduły sprzętowe, przekształcając połączone ze sobą komponenty elektroniczne w działające urządzenie. Oprogramowanie systemu zostało zaimplementowane w postaci zintegrowanej aplikacji w języku Python, w której poszczególne funkcjonalności zostały logicznie wyodrębnione w postaci funkcji. Kluczowym aspektem implementacji była optymalizacja kodu i serwisów działających pod kontrolą systemu operacyjnego Raspberry Pi OS Lite.

2.2.1 Środowisko uruchomieniowe i biblioteki

Ze względu na specyfikę projektu, dobór narzędzi programistycznych podzielono na dwie grupy: biblioteki działające na urządzeniu końcowym (Runtime) oraz narzędzia wykorzystane na stacji roboczej do przygotowania modelu sieci. Jako wiodący język programowania w obu przypadkach wybrano Python w wersji 3.9.

A. Biblioteki systemu wbudowanego (Raspberry Pi)

Kluczowa była kompatybilność bibliotek do obsługi uczenia maszynowego oraz interfejsów sprzętowych. Unikano używania zasobożernych frameworków w celu zapewnienia płynnej pracy na systemie z ograniczoną pamięcią operacyjną – wykorzystano ich wersje zoptymalizowane. W projekcie posłużono się następującymi bibliotekami specjalistycznymi:

- **TensorFlow Lite Runtime (tflite_runtime):** Jest to odchudzona wersja biblioteki TensorFlow. Zawiera wyłącznie interpreter do uruchamiania wytrenowanych modeli. Pozwoliło to oszczędzić kilkaset MB pamięci masowej i znaczące zasoby pamięci operacyjnej w porównaniu do pełnej instalacji pakietu [18].
- **OpenCV (opencv-python-headless):** Biblioteka wykorzystywana do obsługi strumienia wideo z kamery Raspberry Pi Camera Module 3 poprzez sterownik V4L2. Zastosowano wersję *headless* (bez modułów GUI), co redukuje zużycie pamięci RAM w systemie pracującym bez monitora [19].
- **NumPy (numpy):** Podstawowy pakiet do obliczeń numerycznych w języku Python. W projekcie służy między innymi do wykonywania operacji macierzowych na danych obrazu (preprocessing).
- **GPIO Zero (gpiozero):** Wysokopoziomowy interfejs do obsługi pinów GPIO. Wykorzystano moduł Button, który pozwala na implementację obsługi przycisku wielofunkcyjnego w projekcie. Kluczowa w tym celu jest wbudowana obsługa programowania zdarzeniowego (Event-Driven Programming) [20].
- **PySerial (pyserial):** Biblioteka odpowiadająca za obsługę komunikacji dwukierunkowej poprzez porty szeregowe (UART).

B. Biblioteki środowiska treningowego (PC)

Na etapie przygotowania danych i uczenia sieci, gdzie zasoby sprzętowe nie stanowiły ograniczenia, wykorzystano pełne wersje narzędzi badawczych. Posłużono się następującym oprogramowaniem:

- **TensorFlow / Keras:** Pełny framework do uczenia maszynowego. Umożliwił zaprojektowanie architektury sieci, przeprowadzenie procesu trenowania oraz końcową konwersję modelu do formatu obsługiwanego przez system wbudowany.
- **Pillow (PIL):** Biblioteka wykorzystywana do operacji na plikach graficznych. Posłużyła do wczytywania, skalowania oraz augmentacji zdjęć wchodzących w skład zbioru danych treningowych.
- **Seaborn / Matplotlib:** Narzędzia służące do wizualizacji danych statystycznych. W projekcie wykorzystano je do wygenerowania wykresów procesu uczenia oraz macierzy pomyłek obrazującej skuteczność klasyfikacji.

2.2.2 Przygotowanie modelu sieci neuronowej

Proces przygotowania modelu CNN przeprowadzono na zewnętrznej stacji roboczej (komputerze PC) ze względu na znacznie większą moc obliczeniową i wygodę pracy. Cały potok przetwarzania (pipeline) podzielono na cztery etapy:

- Przygotowanie danych (**Dataset**).
- Definicja architektury sieci (**Model**).
- Proces uczenia (**Trening**).
- Optymalizacja (**Kwantyzacja**) do formatu .tflite.

Etap 1: Przygotowanie i czyszczenie zbioru danych

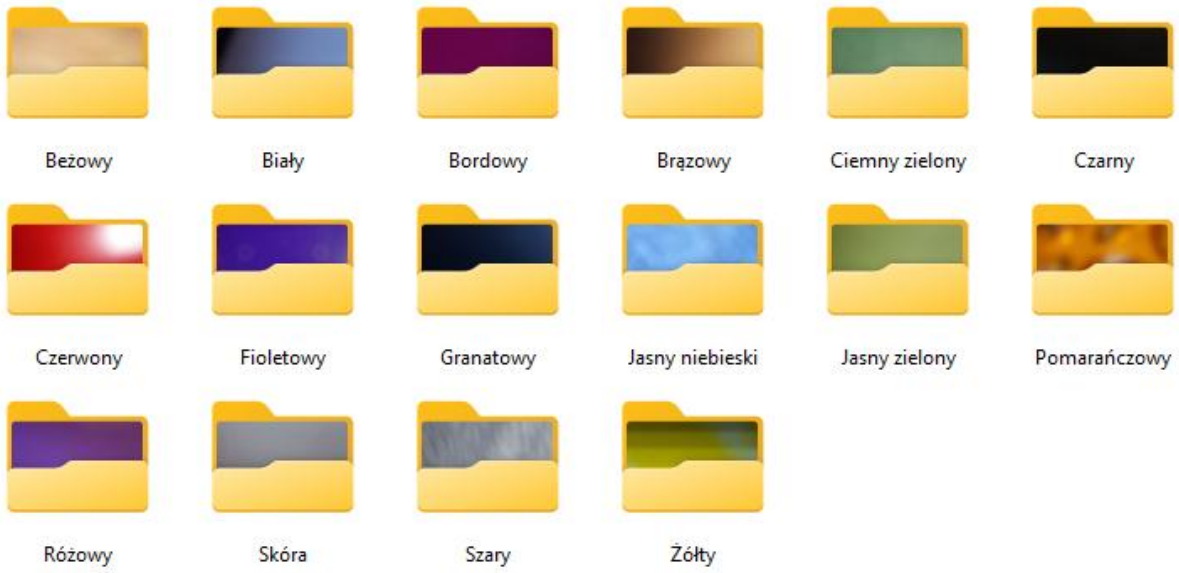
Podstawą każdego skutecznego modelu jest poprawnie sformatowany, odpowiednio rozbudowany oraz zróżnicowany zbiór danych. W ramach pracy wykorzystano autorski zbiór zdjęć podzielony na katalogi odpowiadające nazwom poszczególnych klas (kolorów).

W procesie tworzenia biblioteki obrazów wykorzystano nagrania z kamery Raspberry Pi Camera Module 3, by przystosować algorytm do jej charakterystyki działania. Materiały o różnych teksturach zarejestrowano przy trzech odmiennych temperaturach barwowych oświetlenia, aby zmaksymalizować pokrycie możliwych rzeczywistych warunków użytkowania systemu.

Następnie z materiałów wideo ręcznie wyekstrahowano pojedyncze pliki w formacie JPG, stanowiące podstawę zbioru danych. Aby wyeliminować błędy podczas treningu, opracowano moduł czyszczący (`data_cleaning.py`), realizujący następujące operacje:

- **Standaryzacja formatu:** Konwersja wszystkich plików graficznych do formatu JPG oraz usunięcie metadanych mogących zakłócić proces dekodowania.
- **Weryfikacja integralności:** Skanowanie zbioru biblioteką Pillow (`PIL`) oraz TensorFlow w celu wykrycia i usunięcia plików uszkodzonych, które mogłyby przerwać proces uczenia.

- **Podział zbioru:** Dane zostały automatycznie podzielone na zbiór treningowy (80%) i walidacyjny (20%) z ustalonym ziarnem losowości (seed), co zapewnia powtarzalność eksperymentów.



Rys. 10. Struktura zbioru danych sieci neuronowej

Etap 2: Projekt architektury sieci (Custom CNN).

Zrezygnowano ze stosowania gotowych, rozbudowanych architektur (np. MobileNet), które mogłyby być zbyt obciążające dla Raspberry Pi Zero 2 W, na rzecz zaprojektowania własnej, lekkiej sieci konwolucyjnej (`model.py`). Jej struktura została zoptymalizowana pod kątem rozpoznawania ogólnych cech kolorystycznych, bez względu na teksturę powierzchni, a nie skomplikowanych kształtów. Kluczowe elementy architektury:

- **Warstwa augmentacji danych:** Zintegrowana z modelem lista przekształceń losowych (obrót, odbicie lustrzane, zmiana jasności i kontrastu). Pozwala to na sztuczne poszerzenie zbioru danych o zdjęcia wykonane w warunkach, których nie udało się odwzorować w trakcie tworzenia biblioteki obrazów JPG.

```
# 1. Augmentacja danych
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
    layers.RandomContrast(0.3),
    layers.RandomBrightness(0.3),
], name="data_augmentation")
```

Rys. 11. Kod implementujący augmentacje danych

- **Warstwy konwolucyjne:** Stanowią kluczowy element sieci CNN i odpowiadają za automatyczną ekstrakcję cech z obrazu wejściowego. W zaprojektowanej architekturze

zastosowano trzy kolejne bloki składające się z warstwy konwolucyjnej Conv2D oraz warstwy redukującej rozdzielczość MaxPooling2D z rosnącą liczbą filtrów: 16, 32, 64. Działanie tych funkcji polega na wykonaniu operacji splotu obrazu z zestawem niewielkich filtrów o rozmiarze 3×3. Filtry te uczą się wykrywać najprostsze cechy obrazu, takie jak jednorodne obszary barwne, lokalne zmiany jasności oraz przejścia między kolorami. Zwiększenie liczby filtrów umożliwia modelowi reprezentowanie coraz bardziej złożonych atrybutów obrazu. Następnie warstwa MaxPooling2D redukuje rozdzielczość map cech w celu zmniejszenia złożoności obliczeniowej modelu oraz poprawy jego zdolności do generalizacji [21].

```
# Warstwy Konwolucyjne (Wydobywanie cech)
# Warstwa 1: Wykrywanie podstawowych cech obrazu
# (jednorodne obszary barwne, lokalne przejścia kolorów)
x = layers.Conv2D(16, 3, padding='same', activation='relu')(x)
x = layers.MaxPooling2D()(x)

# Warstwa 2: łączenie prostych cech w bardziej złożone reprezentacje
# (kombinacje kolorów, wpływ oświetlenia)
x = layers.Conv2D(32, 3, padding='same', activation='relu')(x)
x = layers.MaxPooling2D()(x)

# Warstwa 3: Analiza głębsza - zwiększona odporność na cienie
# i zmiany warunków oświetleniowych
x = layers.Conv2D(64, 3, padding='same', activation='relu')(x)
x = layers.MaxPooling2D()(x)
```

Rys. 12. Kod implementujący warstwy konwlucyjne do ekstrakcji cech

Etap 3: Strategia treningowa.

Do uczenia wykorzystano optymalizator Adam, dostosowujący krok uczenia dla poszczególnych wag sieci, przyspieszając i stabilizując proces treningu. Funkcja straty `SparseCategoricalCrossentropy` określa wielkość błędu klasyfikacji, na podstawie którego dzięki wstecznej propagacji błędu w czasie treningu aktualizowane są parametry modelu. Ze względu na nierówną liczbę próbek w poszczególnych klasach, zastosowano automatyczne ważenie klas (class weights). Zapobiega to faworyzowaniu kolorów występujących częściej w bazie. Postęp nauki nadzoruje `ModelCheckpoint`, odpowiadający za zapis wyłącznie wersji modelu osiągającej największą dokładność na zbiorze walidacyjnym.

Etap 4: Konwersja i kwantyzacja

W ostatnim etapie przystosowano model do pracy na procesorze o architekturze ARM bez wydajnego akceleratora GPU. Wykorzystano w tym celu format TensorFlow Lite. W skrypcie zaimplementowano pełną kwantyzację do liczb całkowitych (Full Integer Quantization `INT8`). Proces ten polega na:

- **Kalibracji:** Przepuszczeniu przez model próbki danych ze zbioru treningowego, aby określić zakresy dynamiki aktywacji.

- **Konwersji wag i aktywacji:** Zmianie reprezentacji danych z 32-bitowych liczb zmiennoprzecinkowych (`float32`) na 8-bitowe liczby całkowite (`int8`).

Operacja ta doprowadziła do ok. czterokrotnej redukcji rozmiaru pliku wynikowego oraz znacznie przyspieszyła czas inferencji na Raspberry Pi Zero 2 W. Spadek dokładności zostanie przeanalizowany w kolejnym rozdziale.

2.2.3 Logika sterująca – implementacja skryptu detekcji kolorów na Raspberry Pi

Sercem urządzenia jest skrypt w języku Python integrujący obsługę wszystkich peryferiów oraz wykorzystujący do klasyfikacji kolorów sieć neuronową (CNN). Program opiera się na nieskończonej pętli głównej, której stan pracy (aktywna detekcja lub czuwanie) kontrolowany jest przez przerwania sprzętowe wywoływane przyciskiem monostabilnym. Poniżej przedstawiono kluczowe bloki funkcjonalne zaimplementowane w kodzie:

- **Inicjalizacja i obsługa kamery**

Ze względu na konieczność oszczędzania zasobów (pamięć RAM, bateria), obsługa kamery nie odbywa się w trybie ciągłym, lecz tylko po uruchomieniu aktywnego trybu detekcji kolorów. Zastosowano do tego bibliotekę OpenCV z backendem `CAP_V4L2`.

Kluczowa jest procedura opróżniania bufora przed pobraniem klatki. Raspberry Pi buforuje obraz w pamięci RAM, co przy wolniejszym przetwarzaniu sieci neuronowej powodowałoby analizę klatek sprzed kilku sekund (opóźnienie). Aby temu zapobiec, przed pozyskaniem właściwego zdjęcia programowo opróżniany jest bufor.

```
try:
    # 1. Czyszczenie bufora
    for _ in range(5): cap.grab()

    # 2. Pobranie aktualnej klatki
    t1 = time.time()
    ret, frame = cap.read()
except Exception:
    stop_camera(); start_camera(); continue
```

Rys. 13. Kod implementujący opóźnienie bufora i pobranie aktualnej klatki

Istotnym aspektem konfiguracji sensora jest także sterowanie balansem bieli. Choć w początkowej fazie implementacji korzystano z mechanizmu automatycznego (AWB), w trakcie optymalizacji zrezygnowano z niego. Algorytm AWB dynamicznie zmieniał temperaturę barwową zdjęcia w zależności od tła, co wprowadzało niepożądane przekłamania kolorów, istotne dla sieci neuronowej. Aby poprawić skuteczność klasyfikacji, wyłączono automatykę i ustalono stałe wartości balansu bieli oraz ekspozycji, dobrane w procesie licznych testów.

- **Synteza i odtwarzanie dźwięku**

System komunikacji z użytkownikiem oparto na bibliotece `pygame.mixer`. W przeciwieństwie do klasycznego odtwarzania systemowego za pomocą ścieżki pliku, rozwiązanie to ładuje wszystkie próbki dźwiękowe `.wav` do pamięci RAM podczas inicjalizacji programu. Dzięki temu opóźnienie między wykryciem koloru, a komunikatem głosowym jest minimalne (rzędu milisekund), co jest kluczowe w płynnej interakcji z urządzeniem.

Pliki audio wygenerowano za pomocą sztuczej inteligencji na platformie ElevenLabs [22]. Mikser skonfigurowano na czestotliwość próbkowania 44.1 kHz z buforem 2048 próbek, co stanowi kompromis między jakością dźwięku, a obciążeniem procesora.

- **Obsługa komunikacji UART (M2M)**

Skrypt Python implementuje mechanizm przełączania trybu wyjścia (Głośnik – UART) za pomocą przytrzymania przycisku. W trybie transmisji szeregowej, zamiast generować dźwięk, system buduje ramkę danych w formacie JSON, zawierającą nazwę koloru, poziom pewności (confidence) oraz status detekcji.

```
def report_result(label, confidence, is_uncertain):
    global current_output_mode
    if current_output_mode == "UART":
        data = {
            "color": label,
            "confidence": round(float(confidence), 2),
            "status": "uncertain" if is_uncertain else "ok",
            "timestamp": time.time()
        }
        json_str = json.dumps(data) + "\n"
        if ser and ser.isOpen():
            try:
                ser.write(json_str.encode('utf-8'))
                ser.flush()
                print(f"[UART] Wysłano: {data['color']}")
            except Exception: pass
```

Rys. 14 Kod budujący obiekt JSON

Tak sformatowane dane wysyłane są na wirtualny port szeregowy `/dev/ttyGS0` (skonfigurowany w rozdziale 2.1.5), umożliwiając łatwe parsowanie wyników przez podłączone roboty lub aplikacje diagnostyczne.

- **Obsługa wejść (sterowanie zdarzeniowe)**

Do obsługi przycisku sterującego wykorzystano bibliotekę gpiozero pozwalającą na programowanie zdarzeniowe, eliminując konieczność ciągłego odpytywania stanu pinu (polling) w głównej pętli programu. Zdefiniowano dwie osobne akcje dla jednego fizycznego przycisku – aktywację trybu detekcji kolorów w przypadku krótkiego wciśnięcia oraz zmianę trybu wyjścia (dźwiękowy – UART) w przypadku przytrzymania.

```python
def toggle_state():
    global is_running, was_held
    if was_held:
        was_held = False
        return

    if not is_running:
        print("--- START ANALIZY ---")
        play_sound('analysis_start')
        light.value = LED_BRIGHTNESS
        if start_camera():
            time.sleep(0.8)
            if cap and cap.isOpened():
                for _ in range(3): cap.read()
            is_running = True
        else:
            light.value = 0
    else:
        is_running = False
        light.value = 0
        stop_camera()
        print("--- STOP ANALIZY ---")
        play_sound('analysis_stop')
```

Rys. 15. Kod implementujący aktywację i zatrzymanie trybu detekcji

Parametr `bounce_time=0.2` programowo eliminuje drgania zestyków (debouncing), zapobiegając wielokrotnemu wywołaniu funkcji przy pojedynczym wciśnięciu.

- **Przetwarzanie wyników sieci (Softmax)**

Ostatnim etapem jest interpretacja surowych danych wyjściowych z modelu TFLite. Ponieważ model został skwantyzowany, wartości wyjściowe są liczbami całkowitymi. Aby uzyskać pewność detekcji, zastosowano ręczne obliczenie funkcji Softmax:

```python
# 5. SOFTMAX
raw_logits = output_data[0]
if output_scale > 0:
    logits = (raw_logits.astype(np.float32) - output_zero_point) * output_scale
else:
    logits = raw_logits.astype(np.float32)

exp_logits = np.exp(logits - np.max(logits))
probs = exp_logits / np.sum(exp_logits)

# Decyzja na podstawie aktualnej klatki
prediction_idx = np.argmax(probs)
confidence = probs[prediction_idx]
```

Rys. 16. Kod implementujący obliczanie funkcji Softmax

Wynik porównywany jest ze zdefiniowanym progiem pewności `CONFIDENCE_THRESHOLD`. Jeśli wyliczone prawdopodobieństwo jest od niego niższe, system zwraca komunikat "Niepewny" (lub status `uncertain` w JSON), informując użytkownika, że detekcja może być błędna.

2.2.4 Integracja z systemem operacyjnym (Autostart)

W ostatnim etapie implementacji zapewniono pełną autonomię systemu – możliwość korzystania z urządzenia bez konieczności stałego połączenia Raspberry Pi z zewnętrznym sterownikiem (np. komputerem PC). Posłużono się w tym celu trzema dedykowanymi usługami systemowymi zarządzanymi przez menedżer `systemd`. Utworzono również niezależny serwis kontroli stanu baterii do powiadamiania użytkownika o konieczności naładowania akumulatora.

- **Usługa głównej aplikacji (**`guzik.service`**)**

Plik `/etc/systemd/system/guzik.service` odpowiada za uruchomienie głównego skryptu detekcji kolorów `wykrywacz.py` natychmiast po załadowaniu podstawowych sterowników systemowych. Kluczowe jest przypisanie procesu do grup systemowych audio oraz video (SupplementaryGroups), co gwarantuje bezpośredni dostęp do peryferiów bez uprawnień roota. W poleceniu startowym (`ExecStart`) zastosowano narzędzie pośredniczące `libcamerify`. Zapewnia ono kompatybilność wsteczną biblioteki OpenCV z nowoczesnym stosem sterowników libcamera, stosowanym w systemie Raspberry Pi OS. Stabilność działania zapewnia parametr `Restart=on-failure`, który pełni funkcję programowego watchdoga, automatycznie resetując proces w przypadku wystąpienia nieoczekiwanego błędu.

Kod implementujący `guzik.service`:

```
[Service]
Type=simple
User=admin
Group=audio
SupplementaryGroups=video
ExecStart=/usr/bin/libcamerify /usr/bin/python3
/home/admin/Desktop/wykrywacz.py
Restart=on-failure
```

Listing 1. Fragment kodu implementującego usługę guzik.service

- **Usługa powiadomień systemowych** (`dzwieki_systemowe.service`**)**

Zadaniem tej usługi jest informowanie użytkownika o stanie rozruchu i zamykania systemu operacyjnego. Dzięki opcji `RemainAfterExit=yes`, system traktuje ją jako aktywną po uruchomieniu, co pozwala na wykonanie instrukcji `ExecStop` w momencie zamykania systemu. Wykorzystano tu narzędzie `aplay` skierowane na zdefiniowane wcześniej wirtualne urządzenie `softvol` co gwarantuje odtworzenie dźwięku z ustaloną głośnością.

Start systemu – odtworzenie pliku `on.wav` sygnalizuje użytkownikowi, że system operacyjny jest w pełni gotowy do pracy.

Zamknięcie systemu – odtworzenie pliku `off.wav` następuje w momencie wciśnięcia przycisku zasilania, informując o rozpoczęciu procedury wyłączania.

- **Usługa monitorowania zasilania** (`bateria.service`)

Konieczne było wdrożenie mechanizmu ostrzegawczego działającego w tle, ze względu na użytkownika z dysfunkcją wzroku oraz potrzebę zabezpieczenia ogniwa litowo-polimerowego, które jest wrażliwe na głębokie rozładowanie. Podstawą działania usługi jest skrypt komunikujący się z nakładką UPS HAT poprzez magistralę I2C. W pierwszej kolejności weryfikowana jest obecność układu za pomocą komendy: `sudo i2cdetect -y 1`. Skanuje ona magistralę I2C w poszukiwaniu urządzeń podrzędnych.

```
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- 43 -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

Rys. 17. Lista urządzeń wykrytych na magistrali I2C

Po potwierdzeniu adresu modułu (`0x43`), skrypt pobiera dane z rejestru `0x02` i łączy dwa bajty w jedną wartość 16-bitową. Uzyskaną surową wartość przesunięto bitowo, aby usunąć flagi statusowe.

Zgodnie z dokumentacją układu, rozdzielczość wynosi 0,004 V, co pozwala na wyliczenie napięcia rzeczywistego:

$$\mathrm{V} = \mathrm{Wartość_po_przesunięciu} \times 0{,}004\ [\mathrm{V}] \tag{1}$$

Poziom naładowania przybliżono za pomocą zależności liniowej, przyjmując wartości graniczne dla ogniwa Li-Po: Vmin = 3,0 V (0%) oraz Vmax = 4,15 V (100%). Ostateczny procent wyznaczany jest ze wzoru:

$$\mathrm{Poziom_baterii}\ [\%] = \frac{\mathrm{Vaktualne} - \mathrm{Vmin}}{\mathrm{Vmax} - \mathrm{Vmin}} \times 100\% \tag{2}$$

```
--- Test Waveshare UPS HAT ---

Napięcie: 4.08 V  |  Bateria: 93.6% [##################--]
```

Rys. 18. Wizualizacja działania kodu sprawdzającego poziom baterii

Skrypt ten uruchamiany jest przez `systemd` cyklicznie co 120 sekund, dokonuje szybkiego odczytu i przechodzi w stan uśpienia. Próg krytyczny zdefiniowano na poziomie 3,55 V. Daje on

użytkownikowi około 5 minut działania przed całkowitym wyłączeniem się układu zabezpieczającego BMS. Aby unikać ciągłego powtarzania komunikatu "Niski poziom baterii", zaimplementowano mechanizm cooldown, zapewniający 120 sekund odstępu między kolejnymi alertami dźwiękowymi.

3. Weryfikacja i badania eksperymentalne

Celem niniejszego rozdziału jest weryfikacja poprawności działania skonstruowanego urządzenia oraz skuteczności realizacji założeń projektowych zdefiniowanych na wstępie pracy. Proces badawczy podzielono na dwa główne etapy:

- **Weryfikację funkcjonalną**, mającą na celu potwierdzenie poprawnej współpracy oraz bezawaryjnego działania zintegrowanych modułów.
- **Badania ilościowe**, obejmujące pomiar skuteczności sieci neuronowej, analizę wydajności czasowej systemu i czasu pracy na zasilaniu akumulatorowym.

3.1 Testy funkcjonalne

Celem testów funkcjonalnych była weryfikacja działania komponentów zintegrowanych z oprogramowaniem sterującym. Sprawdzono, czy poszczególne moduły (kamera, audio, zasilanie) reagują prawidłowo na działania użytkownika oraz czy zaimplementowane mechanizmy automatyzacji (usługi `systemd`) działają stabilnie w rzeczywistych warunkach pracy.

Badania przeprowadzono metodą „czarnej skrzynki”, porównując reakcję urządzenia na wymuszenia zewnętrzne (przyciski, odłączenie zasilania, błędy sprzętowe), bez ingerencji w kod źródłowy w czasie trwania testu. Wyniki zestawiono w Tabeli 2.

Tabela 2. Protokół weryfikacji funkcjonalnej urządzenia

Testowana funkcja	Scenariusz testowy	Oczekiwany rezultat	Wynik
Automatyczny rozruch (autostart)	Włączenie zasilania przyciskiem głównym (on/off)	System operacyjny ładuje się, usługi startują w tle, a po załadowaniu słychać komunikat "Urządzenie włączone”.	Pozytywny
Włączenie detekcji koloru	Naciśnięcie przycisku sterującego (GPIO 17).	Urządzenie odczytuje komunikat "Analiza rozpoczęta", po czym zaczyna odczytywać kolory wskazywane przez kamerę.	Pozytywny
Przełączanie trybów (M2M-Audio)	Przytrzymanie przycisku sterującego przez około 2 sekundy.	System odczytuje komunikat "Tryb przewodowy UART" i zaczyna wysyłać dane	Pozytywny

		w formacie JSON do podpiętego komputera PC. Przełączenie powrotne również działa poprawnie.	Poprawność transmisji zweryfikowano za pomocą terminala Putty.
Jakość toru audio	Generowanie komunikatów dźwiękowych w głośnym otoczeniu.	Dźwięk jest wyraźny i komunikaty zrozumiałe, bez zniekształceń przy ustawionej głośności.	Pozytywny
Monitoring zasilania	Praca urządzenia do momentu spadku napięcia akumulatora poniżej 3.55 V.	Wykrycie niskiego poziomu baterii i odtworzenie ostrzeżenia dźwiękowego.	Pozytywny
Bezpieczne wyłączanie	Wciśnięcie przycisku głównego (on/off)	Odtworzenie komunikatu "Wyłączanie" i bezpieczne zamknięcie systemu.	Pozytywny

Przeprowadzone próby potwierdziły, że urządzenie funkcjonuje zgodnie z założeniami projektowymi. Mechanizmy systemowe, w tym serwisy `systemd` oraz obsługa przerwań GPIO poprawnie zarządzają peryferiami, a moduł komunikacji dźwiękowej zapewnia prawidłową interakcję z użytkownikiem.

3.2 Badanie skuteczności sieci neuronowej

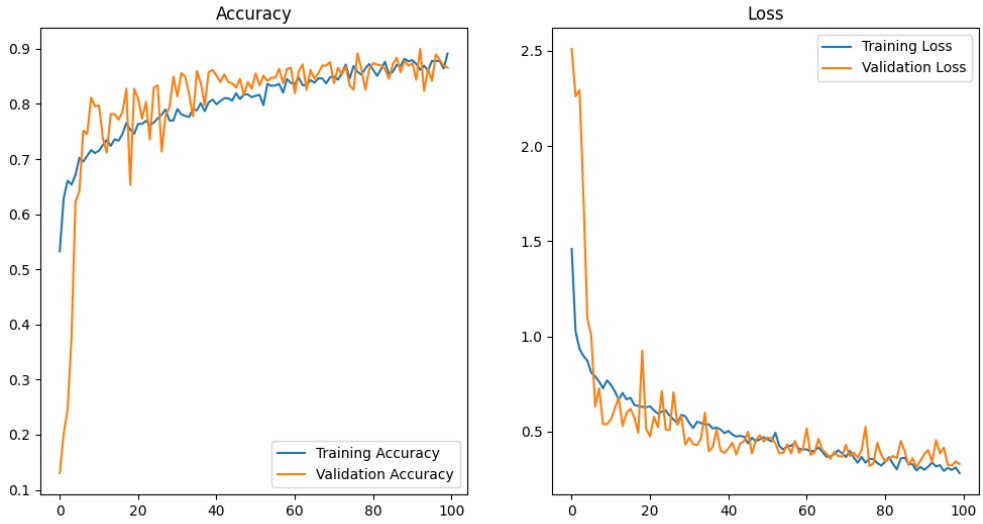
Ocenę jakości zaprojektowanego modelu sieci neuronowej CNN przeprowadzono w dwóch etapach: poprzez analizę metryk treningowych (krzywe uczenia) oraz szczegółową weryfikację błędów klasyfikacji za pomocą macierzy pomyłek. Do badań wykorzystano zbiór walidacyjny, wydzielony przed procesem uczenia i stanowiący 20% całego datasetu. Gwarantuje to pomiary na danych niewykorzystanych bezpośrednio w procesie trenowania modelu.

3.2.1 Analiza przebiegu uczenia

Proces treningu modelu przeprowadzono przez 100 epok, stosując mechanizm ModelCheckpoint do zapisu najlepszych wag. Głównymi wskaźnikami jakości były dokładność (Accuracy) oraz funkcja straty (Loss). Pierwszy z nich informuje o procencie poprawnych

klasyfikacji, a drugi opisuje błąd predykcji, który jest minimalizowany w procesie optymalizacji wag.

Na wykresie przebiegu uczenia **(Rys. 20)** można zaobserwować charakterystyczny wzrost dokładności w początkowych dziesięciu epokach, co wskazuje na szybką adaptację warstw konwolucyjnych do podstawowych cech kolorystycznych obrazu.



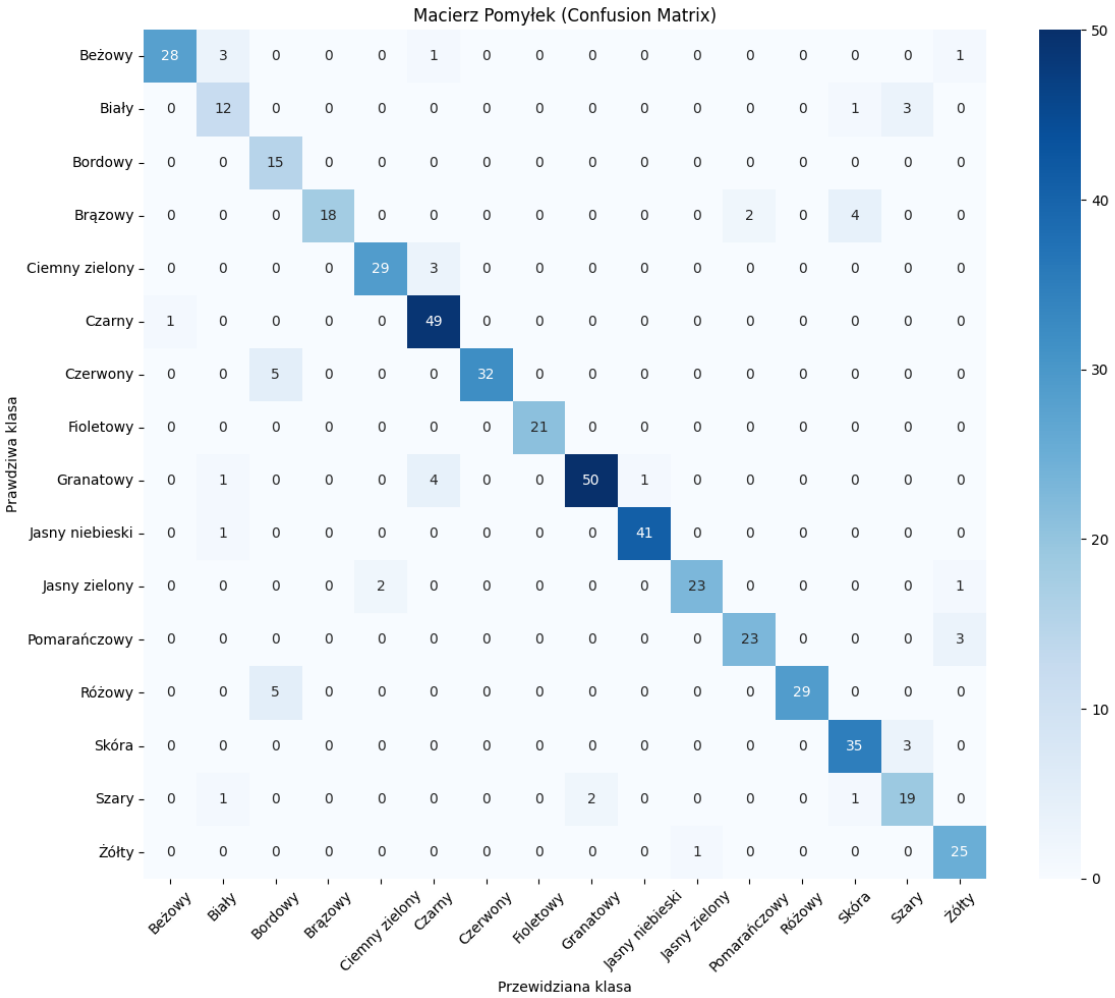
Rys. 19. Wykres zmian dokładności (Accuracy) i funkcji straty (Loss) w funkcji epok dla zbioru treningowego i walidacyjnego

Analiza wykresów pozwala na wyciągnięcie następujących wniosków:

- **Stabilizacja uczenia:** Funkcja straty maleje z upływem kolejnych epok, stabilizując się w końcowej fazie treningu. Potwierdza to poprawny dobór współczynnika uczenia (Learning Rate) optymalizatora Adam.
- **Generalizacja:** Krzywe dla obydwu zbiorów przebiegają blisko siebie. Dopiero w granicach 95. epoki wykres dla danych walidacyjnych przestaje maleć, co oznacza osiągnięcie granicy zdolności generalizacji modelu. Do uzyskania zadowalającego uogólniania cech przyczyniła się intensywna augmentacja danych oraz zastosowanie warstw `Dropout(0.2)`, losowo dezaktywujących 20% neuronów w trakcie uczenia.
- **Wynik końcowy:** Ostateczny model w formacie zmiennoprzecinkowym (`.keras`) osiągnął na zbiorze walidacyjnym dokładność rzędu 89,98% w 93. epoce uczenia.

3.2.2 Szczegółowa analiza błędów (macierz pomyłek, raport klasyfikacji)

Sama wartość dokładności nie określa w pełni charakterystyki działania modelu, dlatego głębszą analizę przeprowadzono za pomocą macierzy pomyłek (Confusion Matrix). Pozwala ona zidentyfikować, które klasy są ze sobą najczęściej mylone.



Rys. 20. Znormalizowana macierz pomyłek dla modelu `.keras` sieci CNN

Na podstawie analizy macierzy zidentyfikowano następujące zależności:

- **Kolory podstawowe:** Klasy o wyraźnie dominującej barwie w przestrzeni RGB, takie jak czerwony, zielony i niebieski, osiągnęły skuteczność detekcji bliską 100%, co świadczy o poprawnej pracy warstw ekstrakcji cech.
- **Barwy problematyczne:** Najwięcej błędów klasyfikacji odnotowano pomiędzy barwami achromatycznymi (biały, szary, czarny). Dodatkowo zidentyfikowano problematyczne pary kolorów, które były ze sobą wzajemnie mylone, w szczególności odcienie z zakresu: różowy-czerwony-bordowy, brązowy-pomarańczowy oraz pary granatowy-czarny i szary-kolor skóry. Wynika to z faktu, że w cyfrowej reprezentacji obrazu kolory te różnią się głównie jasnością (luminancją), a nie chrominancją. W warunkach słabego oświetlenia biały obiekt może zostać zinterpretowany jako szary, a pomarańczowy jako brązowy.
- **Wpływ oświetlenia:** Zauważono, że błędy detekcji często korelują z nienaturalną temperaturą barwową oświetlenia, co uzasadnia programowe wyłączenie automatycznego balansu bieli (AWB) w kamerze, opisane w rozdziale 2.2.3.

Uzupełnieniem macierzy pomyłek jest szczegółowy raport klasyfikacji, przedstawiający skuteczność modelu w kontekście poszczególnych klas z wykorzystaniem trzech kluczowych parametrów:

- **Precyzja (Precision):** Określa wiarygodność modelu, gdy wskazuje on dany kolor. Niska precyzja oznacza częste fałszywe alarmy (False Positives).
- **Czułość (Recall):** Niska czułość oznacza, że model „przegapia" dany kolor (False Negatives), klasyfikując go jako inny.
- **Miara F1 (F1-score):** Średnia harmoniczna precyzji i czułości. Stanowi najlepszą ogólną miarę jakości detekcji dla danej klasy.

```
--- RAPORT KLASYFIKACJI ---
                 precision    recall  f1-score

         Beżowy       0.97      0.85      0.90
          Biały       0.67      0.75      0.71
        Bordowy       0.60      1.00      0.75
        Brązowy       1.00      0.75      0.86
 Ciemny zielony       0.94      0.91      0.92
         Czarny       0.86      0.98      0.92
       Czerwony       1.00      0.86      0.93
      Fioletowy       1.00      1.00      1.00
      Granatowy       0.96      0.89      0.93
Jasny niebieski       0.98      0.98      0.98
  Jasny zielony       0.96      0.88      0.92
   Pomarańczowy       0.92      0.88      0.90
         Różowy       1.00      0.85      0.92
          Skóra       0.85      0.92      0.89
          Szary       0.76      0.83      0.79
          Żółty       0.83      0.96      0.89

       accuracy                           0.90
      macro avg       0.89      0.89      0.89
   weighted avg       0.91      0.90      0.90
```

Rys. 21. Raport klasyfikacji modelu `.keras` sieci CNN

Na podstawie wygenerowanego raportu można sformułować następujące wnioski:

- Model osiągnął ogólną dokładność na poziomie 90%. Dla większości klas, takich jak: Fioletowy (F1=1.00), Jasny niebieski (F1=0.98) czy Ciemny zielony (F1=0.92), sieć wykazuje bardzo wysoką stabilność detekcji.
- Potwierdzają się trudności z rozróżnianiem odcieni szarości (F1=0.79) i bieli (F1=0.71). Klasa Biały osiągnęła stosunkowo niską precyzję (0,67), co oznacza, że model często myli inne jasne lub prześwietlone obiekty z kolorem białym.
- Szczególną uwagę zwraca dysproporcja wyników dla barw czerwonej i bordowej. Czerwony posiada idealną precyzję (1.0), lecz niższą czułość (0,86), co oznacza, że każde wskazanie czerwieni jest poprawne, lecz część jej zdjęć przypisywana jest innym kolorom.

Bordowy wykazuje odwrotną charakterystykę – idealną czułość (1.0), lecz bardzo niską precyzję (0,60). Sugeruje to, że sieć często błędnie identyfikuje inne ciemne barwy (np. ciemnoczerwone lub brązowe) jako kolor bordowy.

Wartość średnia ważona dla miary F1 wynosi 0,90, co jest wynikiem satysfakcjonującym, lecz wskazuje również na możliwość poprawy w dalszym rozwoju modelu.

3.2.3 Wpływ kwantyzacji i konwersji TFLite na jakość detekcji

Kluczowym aspektem pracy była optymalizacja modelu dla platformy Raspberry Pi Zero 2, co sprowadziło się do po treningowej kwantyzacji (`Float32` -> `Int8`) oraz konwersji modelu do fromatu `.tflite`.

```
DOKŁADNOŚĆ MODELU TFLITE: 89.98%

--- RAPORT KLASYFIKACJI (TFLite) ---
                 precision    recall  f1-score

         Beżowy       0.97      0.85      0.90
          Biały       0.63      0.75      0.69
        Bordowy       0.58      1.00      0.73
        Brązowy       1.00      0.79      0.88
 Ciemny zielony       0.94      0.94      0.94
         Czarny       0.88      0.98      0.92
       Czerwony       1.00      0.84      0.91
      Fioletowy       1.00      1.00      1.00
      Granatowy       0.96      0.89      0.93
Jasny niebieski       0.98      0.95      0.96
  Jasny zielony       0.96      0.88      0.92
   Pomarańczowy       0.96      0.88      0.92
         Różowy       1.00      0.85      0.92
          Skóra       0.85      0.92      0.89
          Szary       0.76      0.83      0.79
          Żółty       0.83      0.96      0.89

       accuracy                           0.90
      macro avg       0.89      0.90      0.89
   weighted avg       0.92      0.90      0.90
```

Rys. 22. Raport klasyfikacji skwantyzowanego modelu `.tflite`

Raport klasyfikacji skwantyzowanego modelu wykazał interesujące zjawisko: proces kwantyzacji nie tylko nie spowodował degradacji jakości detekcji, ale w przypadku średniej ważonej (weighted avg) odnotowano wręcz wzrost skuteczności o 1 punkt procentowy. Jest to zjawisko znane w literaturze jako efekt regularyzacyjny kwantyzacji. Redukcja precyzji wag z 32 do 8 bitów zadziałała podobnie do filtru dolnoprzepustowego, usuwając szum numeryczny i nieistotne fluktuacje wag, które mogły powodować nadmierne dopasowanie modelu bazowego do

zbioru treningowego (overfitting). W rezultacie model skwantyzowany charakteryzuje się lepszą generalizacją przy drastycznie mniejszym zapotrzebowaniu na zasoby obliczeniowe.

Pełny proces doprowadził do aż 11-krotnej redukcji rozmiaru pliku wynikowego (z 1,41 MB do zaledwie 125 kB). Oprócz czterokrotnego zmniejszenia wynikającego z samej kwantyzacji (32 bit na 8 bit), konwerter TFLite usuwa także nadmiarowe metadane (stan optymalizatora, gradienty), które są zbędne w procesie inferencji. Wynikowy rozmiar modelu ma kluczowe znaczenie dla wydajności sprzętowej – w pełni mieści się on w 512 kB szybkiej pamięci podręcznej L2 Cache procesora Cortex-A53, wbudowanego w Raspberry Pi Zero 2 W. Sprawia to, że CPU znacznie rzadziej musi sięgać do wolniejszej pamięci RAM, co przekłada się na zauważalną poprawę szybkości działania systemu.

3.2.4 Wydajność czasowa i zasobowa systemu

Kluczowym kryterium użyteczności urządzenia asystującego są czas reakcji oraz stabilność pracy pod obciążeniem. Ze względu na ograniczoną moc obliczeniową platformy Raspberry Pi Zero 2 W, przeprowadzono szereg testów mających na celu weryfikację płynności działania i identyfikację ewentualnych „wąskich gardeł" (bottlenecks).

Tabela 3. Pomiary istotnych parametrów czasowych układu

Parametr	Wartość parametru	Opis pomiaru
Czas rozruchu systemu (boot time)	23s (średnia wartość z 10 pomiarów)	Zmierzono czas od momentu wciśnięcia przycisku włączającego, do usłyszenia komunikatu powitalnego sygnalizującego pełną gotowość do pracy.
Czas pobrania klatki przez kamerę	21 - 35 ms	Programowy odczyt wskaźnika do ostatniej zbuforowanej klatki w pamięci RAM. Czas ten jest krótki, ponieważ nie obejmuje fizycznego czasu naświetlania sensora, który pracuje w trybie ciągłym.
Preprocessing obrazu	5 - 7ms	Czas przycięcia, zmiany rozmiaru, wartości BGR na RGB oraz normalizacji pobranego obrazu.
Inferencja sieci	100 - 130ms	Czas wnioskowania sieci neuronowej. Wykorzystanie wszystkich czterech rdzeni procesora skróciło czas inferencji o ponad

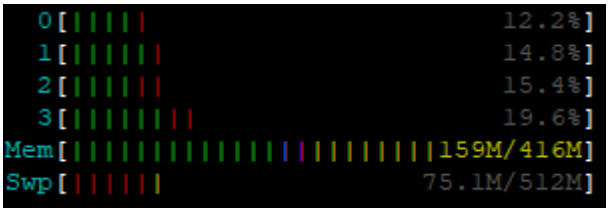
		połowę w porównaniu do pracy skryptu na tylko jednym rdzeniu.
Opóźnienie podsystemu audio	100 - 150ms	Wartość opóźnienia oszacowana została na podstawie architektury biblioteki `pygame` [23]. Zastosowany bufor wyjściowy o wielkości 4096 próbek przy częstotliwości próbkowania 44 100 Hz generuje opóźnienie równe 93 ms. Uwzględniając dodatkowy narzut czasowy sterownika systemowego ALSA, całkowitą latencję podsystemu audio oszacowano na przedział 100–150 ms.
Opóźnienie przetwarzania (End-to-End Latency)	226 - 322ms	Opóźnienie czasowe pełnego procesu od pobrania obrazu po odtworzenie dźwięku odpowiadającego wynikowi klasyfikacji.
Czas pracy urządzenia na zasilaniu akumulatorowym przy aktywnej detekcji	~ 35 min	Zmierzono czas od momentu uruchomienia skryptu detekcji kolorów przy w pełni naładowanym akumulatorze do samoistnego wyłączenia urządzenia po wyczerpaniu energii.

Analiza zbadanych opóźnień czasowych prowadzi do następujących wniosków:

- Całkowite opóźnienie przetwarzania na poziomie 230 do 320 ms jest ledwo odczuwalne podczas użytkowania urządzenia, co daje wrażenie niemalże natychmiastowej reakcji asystenta.
- Najbardziej złożony obliczeniowo proces inferencji sieci neuronowej zajmuje zaledwie ok. 40% całkowitego czasu. Jest to efekt kwantyzacji modelu oraz równoległych obliczeń na 4 rdzeniach procesora.
- Istotnym składnikiem opóźnienia (ok. 45% czasu) pozostaje narzut programowy podsystemu dźwięku. Jest to ograniczenie architektury systemu operacyjnego, które jednak przy obecnym całkowitym czasie reakcji nie wpływa znacząco na komfort użytkowania.

Analiza obciążenia systemu w trybie ciągłej pracy (narzędzie `htop`) wykazała bardzo wysoki poziom optymalizacji oprogramowania. Dzięki rezygnacji z interfejsu graficznego (GUI), zużycie pamięci operacyjnej wynosi zaledwie 159 MB z dostępnych 416 MB. Pozostawia to szeroki margines bezpieczeństwa dla procesów systemowych i eliminuje zjawisko „dławienia się”

(thrashing). W trakcie działania skryptu detekcji kolorów obciążenie rdzeni fluktuuje w granicach 15–20%, co jest bardzo niską wartością. System zachowuje pełną responsywność, obsługując w tle procesy monitorowania baterii i przycisków bez zakłóceń.



Rys. 23. Obciążenie rdzeni procesora i pamięci wizualizowane przez narzędzie `htop`

4. Podsumowanie i wnioski końcowe

Celem niniejszej pracy było zaprojektowanie oraz wykonanie mobilnego systemu do detekcji kolorów dla osób niewidomych i robotów. Na podstawie testów funkcjonalnych i wydajnościowych, a także doświadczeń z pracy nad urządzeniem, zidentyfikowano wady oraz zalety stworzonego prototypu.

4.1 Analiza napotkanych problemów i ograniczeń

W toku prac projektowych oraz testów końcowych zidentyfikowano kilka istotnych ograniczeń technicznych, które znacząco wpłynęły na ostateczne funkcjonowanie prototypu.

4.1.1 Ograniczenia optyczne i oświetleniowe

Największym czynnikiem limitującym skuteczność klasyfikacji kolorów była zmienność warunków oświetleniowych. Obraz z kamery przedstawiający np. białą kartkę znacząco różni się w zależności od tego, czy pada na nią światło o zimnym, czy ciepłym odcieniu. W zależności od temperatury barwowej oświetlenia, pobrana klatka może przyjmować odcień niebieskawy, kremowy, a w warunkach słabego oświetlenia – szary.

Wspomniany w podrozdziale 2.2.3 automatyczny balans bieli (AWB) miał według założeń rozwiązać ten problem. Pomimo znacznej poprawy jakości odwzorowania koloru białego, w przypadku innych barw powodował on jednak istotne przekłamania. Przykładowo, wskazując na czerwony obiekt pod ciepłym światłem lampy, algorytm AWB potrafił błędnie skorygować obraz w kierunku barwy różowej. Spowodowało to konieczność wyłączenia automatyki i zastosowania stałych wartości parametrów. Aby skompensować brak aktywnego balansu bieli, podczas tworzenia bazy danych dla sieci neuronowej fotografowano obiekty przy zróżnicowanych temperaturach barwowych oświetlenia. Pomogło to częściowo zniwelować problem, lecz spowodowało większe podobieństwo cech niektórych klas (np. białego i kremowego), co nadal skutkuje sporadycznymi błędami w klasyfikacji.

4.1.2 Zarządzanie zasobami pamięci RAM i CPU

Pomimo wstępnych obaw o ograniczone zasoby platformy Raspberry Pi Zero 2 W, dostępne 512 MB pamięci RAM oraz czterordzeniowy procesor wykorzystywane są w stopniu umiarkowanym. Dzięki optymalizacji i skwantowaniu modelu CNN do formatu TFLite, proces detekcji zajmuje jedynie ok. 30% pamięci operacyjnej, a średnie obciążenie jednostki centralnej nie przekracza 20%. Główną zaletą zastosowanego CPU było skrócenie czasu inferencji poprzez

zrównoleglenie obliczeń. Pozostała część zasobów stanowi zapas mocy, co pozwala na potencjalną rozbudowę oprogramowania w przyszłości bez konieczności wymiany sprzętu.

4.1.3 Opóźnienia podsystemu audio i ładowanie systemu operacyjnego

Analiza wydajnościowa wykazała, że obsługa dźwięku stanowi istotny czynnik wpływający na całkowite opóźnienie systemu. Konieczność buforowania próbek dźwiękowych przez system operacyjny Linux wprowadza stałą latencję rzędu 100–150 ms. Jest to ograniczenie architektury sterownika ALSA, trudne do wyeliminowania bez zastosowania dedykowanych systemów czasu rzeczywistego (RTOS). Kolejną wadą zaprojektowanego rozwiązania jest długi czas rozruchu urządzenia (ponad 20 s), zmierzony w rozdziale 3.4. Wynika on z konieczności załadowania pełnego jądra systemu, inicjalizacji sterowników sprzętowych oraz usług systemowych, co stanowi zauważalny narzut czasowy w porównaniu do prostszych rozwiązań mikrokontrolerowych.

4.2 Możliwość dalszego rozwoju projektu

Zrealizowany prototyp stanowi znakomitą bazę do dalszego rozwoju. Architektura oparta na skryptach w języku Python na minikomputerze Raspberry Pi daje szerokie możliwości rozbudowy systemu. Na podstawie wyciągniętych wniosków wyznaczono potencjalne kierunki usprawnień urządzenia w przyszłych iteracjach:

- **Integracja oświetlenia aktywnego**

Aby uniezależnić urządzenie od warunków oświetleniowych lub znacząco zniwelować ich wpływ, zalecane jest dodanie diody Power LED o wysokim współczynniku oddawania barw (CRI), sterowanej bezpośrednio z pinu GPIO. Pozwoliłoby to na doświetlanie obrazu w ciemnym otoczeniu i potencjalnie ustabilizowanie odcienia bieli w skrajnych przypadkach temperatur oświetlenia.

- **Profesjonalizacja warstwy sprzętowej (PCB)**

Obecna wersja urządzenia oparta jest na ręcznie lutowanych połączeniach przewodowych, co jest wystarczające dla prototypu, ale podatne na uszkodzenia mechaniczne. Kolejnym krokiem inżynierskim powinno być zaprojektowanie dedykowanej płytki drukowanej (PCB) typu „HAT”, integrującej przyciski i wyprowadzenia audio. Zwiększyłoby to niezawodność i pozwoliło na miniaturyzację urządzenia.

- **Rekonfiguracja dobranych elementów systemu**

Po przeprowadzonych testach oraz doświadczeniach z procesu implementacji, możliwa jest optymalizacja doboru podzespołów. Moduły podsystemu audio (wzmacniacz MAX98357A, głośnik YD40-18) i zasilanie UPS HAT sprawują się bezbłędnie. Natomiast istnieje możliwość, że

pozostałe elementy mogą być zastąpione przez bardziej optymalne rozwiązania. Brak wykorzystania AWB w Raspberry Pi Camera Module 3 sprawia, iż model poprzedniej generacji może stać się bardziej korzystną cenowo opcją. Podobnie platforma główna Raspberry Pi Zero 2 W, ze względu na niskie wykorzystanie zasobów (pamięci RAM i CPU), może zostać zastąpiona tańszą i mniejszą alternatywą.

- **Stworzenie obszernej bazy danych treningowych dla sieci neuronowej**

Stworzona i wykorzystana w procesie treningu CNN baza składa się łącznie z 2500 obrazów (100–200 na klasę) przedmiotów o różnych kolorach i teksturach. Zdjęcia wykonywano pod trzema różnymi odcieniami światła z lampy LED w celu dywersyfikacji datasetu oraz chociaż częściowego pokrycia rzeczywistych warunków. Nie zmienia to faktu, że zbiór ten jest mikroskopijny w porównaniu do standardowych baz danych wykorzystywanych w profesjonalnych aplikacjach sieci neuronowych. Przy dalszym rozwoju pracy zaleca się powiększyć zbiór obrazów co najmniej kilkukrotnie oraz pozyskać je bezpośrednio w środowiskach wykorzystania systemu (np. na zewnątrz, w sklepach).

4.3 Wnioski końcowe

Główny cel pracy, jakim było stworzenie mobilnego asystenta rozpoznawania kolorów dla osób niewidomych z opcją wykorzystania w systemie wizyjnym robota, został osiągnięty. Zaprojektowane urządzenie charakteryzuje się:

- **Wysoką skutecznością:** Osiągnięto dokładność klasyfikacji na poziomie 90%, co jest wynikiem w pełni użytecznym w codziennych zastosowaniach.
- **Działaniem w czasie rzeczywistym:** Dzięki kwantyzacji modelu oraz optymalizacji procesu detekcji (m.in. poprzez wykorzystanie wielowątkowości), całkowity czas reakcji systemu wynosi średnio 230–320 ms. Jest to wartość mieszcząca się w granicach percepcji natychmiastowej i zapewnia wysoki komfort użytkowania.
- **Niskim kosztem implementacji:** Całkowity koszt komponentów wykorzystanych w ostatecznej wersji prototypu wyniósł około 350 zł. Sprawia to, że zbudowane urządzenie jest wielokrotnie tańsze od obecnych na rynku odpowiedników, stając się rozwiązaniem dostępnym dla szerszego grona odbiorców.
- **Energooszczędnością i mobilnością:** Optymalizacja oprogramowania pozwoliła na efektywne wykorzystanie platformy Raspberry Pi Zero 2 W, tworząc urządzenie lekkie i zdolne do długiej pracy na zasilaniu akumulatorowym.

Bibliografia:

[1] Główny Urząd Statystyczny, „Dzień Białej Laski – ponad 160 tysięcy Polaków żyje z chorobami wzroku”, [Online]. Dostępne: https://stat.gov.pl/dla-mediow/informacje-prasowe/dzien-bialej-laski-ponad-160-tysiecy-polakow-zyje-z-chorobami-wzroku-az-4-na-10-z-nich-ma-znaczny-stopien-niepelnosprawnosci,33,1.html. [Dostęp: 10.12.2025].

[2] World Health Organization, „Visual impairment and blindness”, [Online]. Dostępne: https://www.who.int/news/item/09-10-2003-up-to-45-million-blind-people-globally---and-growing. [Dostęp: 10.12.2025].

[3] CareTec GmbH, „Colorino - Color and Light Detector”, [Online]. Dostępne: https://www.caretec.at/product/colorino-color-and-light-detector/. [Dostęp: 13.12.2025].

[4] Altix Sp. z o.o., Oferta produktów Colorino, [Online]. Dostępne: http://www.sklep.altix.pl/pl/search?q=colorino. [Dostęp: 13.12.2025].

[5] ScienceDirect, Multilayer Perceptron, [Online]. Dostępne: https://www.sciencedirect.com/topics/computer-science/multilayer-perceptron. [Dostęp: 9.10.2025].

[6] IBM, „What is a neural network?”, [Online]. Dostępne: https://www.ibm.com/think/topics/neural-networks. [Dostęp: 9.10.2025].

[7] R. Kaur, „Decoding CNNs: a Beginner’s Guide to Convolutional Neural Networks and their applications”, Medium.com, [Online]. Dostępne: https://ravjot03.medium.com/decoding-cnns-a-beginners-guide-to-convolutional-neural-networks-and-their-applications-1a8806cbf536. [Dostęp: 9.10.2025].

[8] Espressif Systems, „ESP32-S3 Technical Reference Manual”, [Online]. Dostępne: https://documentation.espressif.com/esp32-s3_technical_reference_manual_en.pdf. [Dostęp: 20.05.2025].

[9] TronicsBench, „Using the TP4056”, [Online]. Dostępne: https://www.best-microcontroller-projects.com/tp4056.html. [Dostęp: 25.05.2025].

[10] Phipps Electronics, „An Introduction to I2S Audio”, [Online]. Dostępne: https://www.phippselectronics.com/an-introduction-to-i2s-audio/. [Dostęp: 25.05.2025].

[11] Analog Devices, „UART: a Hardware Communication Protocol”, [Online]. Dostępne: https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html. [Dostęp: 22.10.2025].

[12] Raspberry Pi Ltd, „Raspberry Pi computer hardware”, [Online]. Dostępne: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html. [Dostęp: 25.05.2025].

[13] Raspberry Pi Ltd, „Getting started with your Raspberry Pi”, [Online]. Dostępne: https://www.raspberrypi.com/documentation/computers/getting-started.html. [Dostęp: 25.05.2025].

[14] Raspberry Pi Ltd, „Camera software”, [Online]. Dostępne: https://www.raspberrypi.com/documentation/computers/camera_software.html. [Dostęp: 28.05.2025].

[15] Waveshare, „UPS HAT (C)”, [Online]. Dostępne: https://www.waveshare.com/wiki/UPS_HAT_(C. [Dostęp: 28.05.2025].

[16] Analog Devices, "MAX98357A/MAX98357B: Tiny, Low-Cost, PCM Class D Amplifier with Class AB Performance”, [Online]. Dostępne: https://www.analog.com/media/en/technical-documentation/data-sheets/max98357a-max98357b.pdf. [Dostęp: 28.05.2025].

[17] The Linux Kernel Organization, „USB Gadget API for Linux”, [Online]. Dostępne: https://www.kernel.org/doc/html/v4.19/driver-api/usb/gadget.html. [Dostęp: 22.10.2025].

[18] Google Brain Team, „TensorFlow v2.16.1”, [Online]. Dostępne: https://www.tensorflow.org/api_docs/python/tf. [Dostęp: 9.10.2025].

[19] OpenCV Team, „OpenCV modules”, [Online]. Dostępne: https://docs.opencv.org/4.x/. [Dostęp: 28.05.2025].

[20] gptiozero.readthedocs.io, „gpiozero” [Online]. Dostępne: https://gpiozero.readthedocs.io/en/stable/. [Dostęp: 02.11.2025].

[21] Joaquin Vanschoren, Eindhoven University of Technology, „Convolutional Neural Networks”, [Online]. Dostępne: https://ml-course.github.io/master/09%20-%20Convolutional%20Neural%20Networks.pdf. [Dostęp: 9.10.2025].

[22] ElevenLabs Inc., "Text to Speech & AI Voice Generator," [Online]. Dostępne: https://elevenlabs.io/. [Dostęp: 25.11.2025].

[23] Pygame Community, „Pygame Mixer Module Documentation”, [Online]. Dostępne: https://www.pygame.org/docs/ref/mixer.html. [Dostęp: 25.11.2025].

Spis Ilustracji

Rys. 1. Topologia sieci MLP [5] 10
Rys. 2. Schemat Konwolucyjnej Sieci Neuronowej [7] 10
Rys. 3. Schemat układu z modułem ładowania TP4056 [9] 13
Rys. 4. Diagram czasowy dla linii I2S [10] 14
Rys. 5. Schemat komunikacji dwóch interfejsów UART [11] 15
Rys. 6. Schemat blokowy systemu detekcji kolorów dla osób niewidomych i robotów 16
Rys. 7. Schemat blokowy połączeń wzmacniacza MAX98357A z Raspberry Pi Zero 2 W 19
Rys. 8. Zawartość pliku konfiguracyjnego /etc/asound.conf 20
Rys. 9. Wirtualny suwak do kontroli poziomu sygnału elementu `Master` 21
Rys. 10. Struktura zbioru danych sieci neuronowej 25
Rys. 11. Kod implementujący augmentacje danych 25
Rys. 12. Kod implementujący warstwy konwlucyjne do ekstrakcji cech 26
Rys. 13. Kod implementujący opóźnienie bufora i pobranie aktualnej klatki 27
Rys. 14 Kod budujący obiekt JSON 28
Rys. 15. Kod implementujący aktywację i zatrzymanie trybu detekcji 29
Rys. 16. Kod implementujący obliczanie funkcji Softmax 29
Listing 1. Fragment kodu implementującego usługę guzik.service 30
Rys. 17. Lista urządzeń wykrytych na magistrali I2C 31
Rys. 18. Wizualizacja działania kodu sprawdzającego poziom baterii 31
Rys. 19. Wykres zmian dokładności (Accuracy) i funkcji straty (Loss) w funkcji epok dla zbioru treningowego i walidacyjnego 35
Rys. 20. Znormalizowana macierz pomyłek dla modelu `.keras` sieci CNN 36
Rys. 21. Raport klasyfikacji modelu `.keras` sieci CNN 37
Rys. 22. Raport klasyfikacji skwantyzowanego modelu `.tflite` 38
Rys. 23. Obciążenie rdzeni procesora i pamięci wizualizowane przez narzędzie `htop` 41

Spis Tabel

Tabela 1. Konfiguracja podłączenia przycisków sterujących 22

Tabela 2. Protokół weryfikacji funkcjonalnej urządzenia 33

Tabela 3. Pomiary istotnych parametrów czasowych układu 39